

An end-to-end deep learning method for solving nonlocal Allen–Cahn and Cahn–Hilliard phase-field models[☆]

Yuwei Geng^a, Olena Burkovska^b , Lili Ju^a , Guannan Zhang^b ,^{*}
Max Gunzburger^{c,d}

^a Department of Mathematics, University of South Carolina, Columbia, SC, 29208, USA

^b Computer Science and Mathematics Division, Oak Ridge National Laboratory, Oak Ridge, TN, 37831, USA

^c Department of Scientific Computing, Florida State University, Tallahassee, FL, 32306, USA

^d Oden Institute for Computational Engineering and Sciences, University of Texas at Austin, Austin, TX, 78712, USA

ARTICLE INFO

Keywords:

Neural network
Nonlocal models
Phase-field models
Deep learning
Allen–Cahn equation
Cahn–Hilliard equation
Non-smooth problems

ABSTRACT

We propose an efficient end-to-end deep learning method for solving nonlocal Allen–Cahn (AC) and Cahn–Hilliard (CH) phase-field models. One motivation for this effort emanates from the fact that discretized partial differential equation-based AC or CH phase-field models result in diffuse interfaces between phases, with the only recourse for remediation is to severely refine the spatial grids in the vicinity of the true moving sharp interface whose width is determined by a grid-independent parameter that is substantially larger than the local grid size. In this work, we introduce non-mass conserving nonlocal AC or CH phase-field models with regular, logarithmic, or obstacle double-well potentials. Because of non-locality, some of these models feature totally sharp interfaces separating phases. The discretization of such models can lead to a transition between phases whose width is only a single grid cell wide. Another motivation is to use deep learning approaches to ameliorate the otherwise high cost of solving discretized nonlocal phase-field models. To this end, loss functions of the customized neural networks are defined using the residual of the fully discrete approximations of the AC or CH models, which results from applying a Fourier collocation method and a temporal semi-implicit approximation. To address the long-range interactions in the models, we tailor the architecture of the neural network by incorporating a nonlocal kernel as an input channel to the neural network model. We then provide the results of extensive computational experiments to illustrate the accuracy, predictive capabilities, and cost reductions of the proposed method.

1. Introduction

Phase-field modeling is a robust method utilized to describe a wide range of physical processes, including nucleation and solidification in materials science [1,2], fracture [3], image processing [4,5]. These models, particularly local phase-field models of the Allen–Cahn or Cahn–Hilliard type, rely on local partial derivatives and are commonly employed due to their versatility.

[☆] This manuscript has been authored by UT-Battelle, LLC, under contract DE-AC05-00OR22725 with the US Department of Energy (DOE). The US government retains and the publisher, by accepting the article for publication, acknowledges that the US government retains a nonexclusive, paid-up, irrevocable, worldwide license to publish or reproduce the published form of this manuscript, or allow others to do so, for US government purposes. DOE will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan.

^{*} Corresponding author.

E-mail addresses: ygeng@email.sc.edu (Y. Geng), burkovskao@ornl.gov (O. Burkovska), ju@math.sc.edu (L. Ju), zhangg@ornl.gov (G. Zhang), max.gunzburger@austin.utexas.edu (M. Gunzburger).

<https://doi.org/10.1016/j.cma.2024.117721>

Received 4 October 2024; Received in revised form 6 December 2024; Accepted 30 December 2024

0045-7825/© 2024 Published by Elsevier B.V.

Nevertheless, a significant limitation of these models is their tendency to produce diffuse interfaces which poses computational challenges, especially when thin or sharp interfaces are intrinsic features of the models. For instance, in material solidification applications the interface between two immiscible substances is typically represented by a moving sharp interface that separates the distinct phases. In practice, implementing such models is difficult due to the complexities associated with the coupling at the moving interface. Instead, a phase-field method can be adopted wherein the sharp interface is approximated by a diffuse interface. This approach eliminates the need for explicit interface tracking and allows for the simulation of the entire phase transition process. However, accurately resolving very thin diffuse interfaces to approximate a sharp interface necessitates the use of very fine meshes and/or more structurally complex models, which greatly increases the computational demands.

To ameliorate these deficiencies of local phase-field models based on partial differential equations (PDEs), we consider *nonlocal phase-field models*. The essential difference between nonlocal phase-field models and local phase-field models is that the former involves a derivative-free integral operator that allows for pairs of points separated by a finite distance to interact with each other, in contrast to PDE models for which pairs of points interact only within the infinitesimal neighborhoods needed to define derivatives. Nonlocal models lead to improved physical realism through a better representation of long-range interactions and correlations, enhanced numerical stability when dealing with sharp interfaces, and superior multi-scale modeling capabilities. Nonlocal models also reduce mesh dependencies, more easily incorporate statistical effects and heterogeneities, and provide a more accurate representation of interfacial phenomena. These features collectively result in a more comprehensive and physically accurate framework for simulating complex material systems, particularly those involving multiple phases, long-range interactions, and multi-scale phenomena. On the other hand, the nonlocal phase-field model is much more difficult to solve numerically due to the existence of the nonlocal integral operator, which makes deep learning methods an attractive alternative.

The practical applications of phase-field models in materials science and engineering demand that deep learning models for AC or CH equations possess the capability to accurately and efficiently capture moving sharp interfaces across a diverse range of initial conditions. This requirement stems from the need to simulate complex phase transformations and microstructural evolution in real-world materials systems. The challenge lies in developing neural network architectures that can not only handle the nonlinear dynamics of these equations but also maintain high fidelity in representing sharp interfaces without excessive computational cost. Moreover, these models must be generalizable enough to perform well on various initial configurations, including those not seen during training. Despite significant progress in applying machine learning to PDEs, achieving this ideal balance of accuracy, efficiency, and versatility for phase-field models remains an open challenge in the field, with current solutions often trading off one aspect for another. For example, the standard Physics-Informed Neural Networks (PINNs) face significant challenges in accurately resolving sharp interfaces, as noted by [6–9]. These challenges arise due to the neural network's inherent tendency to smooth out steep gradients. Adaptive sampling methods were proposed to dynamically select collocation points in regions with steep gradients. For example, [10] proposed an adaptive sampling strategy that selects collocation points and implements a time-adaptive strategy within sub-time intervals to capture sharp transitions when solving AC and CH equations, but this method cannot generalize to a broad range of initial conditions. Operator Learning-enhanced PINN (OL-PINN) [11] involves pre-training a deep neural network (DeepONet) on a class of smooth problems before integrating them with PINNs to better capture sharp interface features. However, the PINN model still needs to be retrained for each initial condition, which makes it computationally expensive for the nonlocal phase-field model under consideration. Alternative approaches based on neural operators and their generalizations [12–15] have been actively explored in the literature. Those methods, often do not require the knowledge of the underlying physics and are based on a data-driven loss. In [16], the authors present a framework that combines a convolutional autoencoder architecture with a DeepONet to learn the microstructure evolution of the phase-field variable. In contrast to our approach, this method employs a more complex architecture and is entirely data-driven, as it relies on a data-driven loss. To increase the accuracy a hybrid approach is also adopted.

Neural networks proposed by [17] emulate the action of diffusion and reaction operators, where convolutional layers approximate the diffusion process and multilayer perceptrons capture nonlinear reaction dynamics, enabling efficient simulations of interface evolution in AC equations. Similarly, “Phase-Field DeepONet” [18] is a physics-informed deep operator neural network framework designed for rapid simulations of pattern formation in systems governed by gradient flows, validated on the local AC and CH equations. Although these methods leverage operator learning and do not require retraining for different initial conditions, they are not directly applicable to nonlocal problems, which involve interactions across distant points in the domain.

In this paper, we propose a more general framework, referred to as the *nonlocal phase-field network* (NPF-Net) for solving nonlocal AC and CH equations with both smooth and sharp interfaces. Building on our previous work in [19], NPF-Net aims to learn fully discrete nonlocal operators that achieve numerical approximation accuracy between adjacent time steps for the target equations. The loss functions are formulated using a residual discretized by a Fourier collocation method in space with first- or second-order semi-implicit time stepping schemes. In this case, no training data needs to be generated a priori to train the NPF-Net model. The key contributions of this work are threefold. First, we present novel network architectures tailored for solving nonlocal systems, with a specific focus on nonlocal AC and CH equations. This design enables more accurate and efficient simulations of long-range interactions in materials. Second, our model successfully handles equations with non-smooth potential functions, a challenging aspect in both traditional numerical methods and machine learning approaches. This capability allows for more realistic modeling of complex material behaviors. Lastly, and perhaps most significant, NPF-Net demonstrates the ability to accurately describe sharp interface dynamics, a critical feature for simulating phenomena such as phase boundaries and other discontinuous processes in materials.

The remainder of the paper is organized as follows. In Section 3, we introduce space and time discretization methods for the nonlocal AC and CH equations. We then discuss, in Section 4, the loss function, training strategy, and network architectures used

for simulating the dynamics of the AC and CH equations using deep learning methods. Hyperparameter optimization, numerical experiments, and comparisons are presented in Section 5 which demonstrate the superior performance of our proposed method, including an evaluation of its prediction and generalization capabilities under various scenarios in two and three dimensions. Finally, concluding remarks are provided in Section 6.

2. Nonlocal Allen-Cahn and Cahn-Hilliard phase-field models

In this section, we provide a brief introduction of the nonlocal AC and CH models for three types of potential functions and the use of deep learning strategies to lessen the costs of computational simulations.

Let $\Omega \subset \mathbb{R}^d$, $d = 2, 3$, be a bounded domain. Specifically, we consider the nonlocal problem given by

$$\begin{cases} \partial_t u + Aw = 0, \\ w = B_\delta u + \partial F(u), \end{cases} \quad (2.1)$$

and equipped with periodic boundary conditions and a suitable initial condition $u(0, x) = u_0(x)$. Here, B_δ denotes a nonlocal diffusion operator defined as

$$B_\delta u := \int_{\Omega} \gamma_\delta(x - y)(u(x) - u(y)) dy = (\gamma_\delta * 1)u(x) - (\gamma_\delta * u)(x) = c_{\gamma_\delta}u - \gamma_\delta * u,$$

where $\gamma_\delta(y - x)$ is a nonlocal interaction kernel, parameterized by the positive horizon parameter $\delta > 0$ measuring the effective range of nonlocal interactions. The nonlocal kernel is assumed to be integrable, positive, and symmetric (i.e., $\gamma_\delta(y - x) = \gamma_\delta(x - y)$) with $c_{\gamma_\delta} = \gamma_\delta * 1 = \int_{\Omega} \gamma_\delta(x - y) dx$ and with $*$ denoting the convolution operator. We define the operator $A = I - \beta \Delta$ with $\beta \geq 0$, where $\beta = 0$ corresponds to a non-mass conserving nonlocal AC model and $\beta > 0$ to a non-mass conserving nonlocal CH model. Under appropriate conditions on the kernel, we can recover a local operator in the limit of vanishing nonlocal interactions, as $\delta \rightarrow 0$ i.e., $B_\delta u \rightarrow -\epsilon^2 \Delta u$ with a local interface parameter $\epsilon^2 = \frac{1}{2n} \int_{\Omega} |\zeta|^2 \gamma_\delta(|\zeta|) d\zeta$ (see, e.g., [20]). We also define an associated nonlocal energy, which is an analogous of the local Ginzburg–Landau energy:

$$E(u) = \frac{1}{4} \int_{\Omega} \int_{\Omega} (u(x) - u(y))^2 \gamma_\delta(x - y) dx dy + \int_{\Omega} F(u(x)) dx. \quad (2.2)$$

In (2.1) and (2.2), F denotes a double-well potential with ∂F denoting a generalized subdifferential of F . We consider the three common types of potentials, namely the regular, logarithmic, and obstacle potentials. Those potentials can be expressed as

$$F(u) = \frac{c_F}{2}(1 - u^2) + \psi(u), \quad c_F > 0, \quad (2.3)$$

where a *smooth regular potential* $\psi(u)$ is defined as

$$\psi_{\text{reg}}(u) = \frac{c_F}{4}(u^4 - 1), \quad c_F > 0, \quad (2.4)$$

a *logarithmic potential* is defined as, for $0 < \theta < c_F$,

$$\psi_{\log}(u) = \begin{cases} \frac{\theta}{2}((1 + u)\ln(1 + u) + (1 - u)\ln(1 - u)), & \text{if } u \in [-1, 1], \\ +\infty, & \text{else,} \end{cases} \quad (2.5)$$

where the values at $u = \pm 1$ are defined by taking a limit of the function resulting in $\theta_c \ln(2)$. Eventually, a *non-smooth obstacle potential* is defined as the convex indicator function

$$\psi_{\text{obs}}(u) = \mathbb{I}_{[-1, 1]}(u) = \begin{cases} 0, & \text{if } u \in [-1, 1], \\ +\infty, & \text{else.} \end{cases} \quad (2.6)$$

The corresponding differential or subdifferential, as the case may be, of $F(u)$ is then defined as

$$\partial F(u) = -c_F u + \partial \psi(u), \quad (2.7)$$

with $\partial \psi(u)$ given by

$$\partial \psi_{\text{reg}}(u) = \psi'(u) = \{c_F u^3\}, \quad (2.8a)$$

$$\partial \psi_{\log}(u) = \begin{cases} \emptyset, & \text{if } u \notin [-1, 1], \\ \left\{ \frac{\theta_c}{2} (\ln(1 + u) - \ln(1 - u)) \right\}, & \text{if } u \in (-1, 1), \end{cases} \quad (2.8b)$$

$$\partial \psi_{\text{obs}}(u) = \partial \mathbb{I}_{[-1, 1]}(u) = \begin{cases} (-\infty, 0], & \text{if } u = -1, \\ 0, & \text{if } u \in (-1, 1), \\ [0, \infty), & \text{if } u = 1. \end{cases} \quad (2.8c)$$

We note that for the case of regular and logarithmic potentials ($u \in (-1, 1)$) the subdifferential $\partial \psi$ reduces to a classical derivative of those potentials. Illustrations of the three potential functions and their subderivatives or subdifferentials are given in Fig. 2.1.

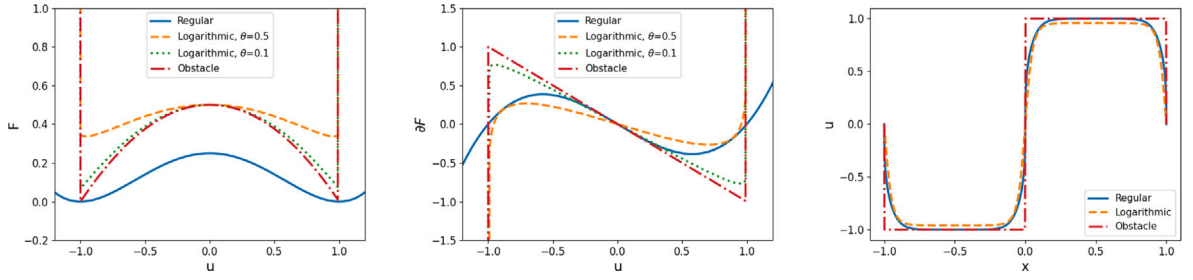


Fig. 2.1. The plots of the different potential functions (left), their corresponding subdifferentials (derivatives) (middle), and the solutions of (2.1) corresponding to those potentials. Here we use $c_F = 1$, $\xi = c_{\gamma_\delta} - c_F = 0$, and the solution (right) for the logarithmic potential is depicted for $\theta_c = 0.5$.

Using the above definitions for the subderivative or subdifferential, we can rewrite the model in (2.1) as¹

$$\begin{cases} \partial_t u + Aw = 0, \\ w - \xi u + \gamma_\delta * u - \partial\psi(u) = 0, \end{cases} \quad (2.9)$$

where $\xi = c_{\gamma_\delta} - c_F$ is a nonlocal interface parameter and plays an analogous role to that of the local interface parameter ϵ when $B_\delta u \approx -\epsilon^2 \Delta u$. Here, larger values of ξ result in a more diffuse interface, as noted in [21]. However, unlike the local setting for which a sharp interface is achieved only in the limit $\epsilon \rightarrow 0$, in nonlocal settings well-defined sharp interfaces with distinct pure phases, i.e., leading to a piece-wise constant solution profile with no mixing, can emerge when $\xi = 0$ [21]. Alternatively, we can set $w = -A^{-1}(\partial_t u) = G(\partial_t u)$, where G denotes the Green's function related to the operator A , so that then we obtain

$$-G(\partial_t u) + \xi u - \gamma_\delta * u + \partial\psi(u) = 0. \quad (2.10)$$

The choice of potential greatly affects the properties of the solution such as interface sharpness. The obstacle potential produces the sharpest interface, whereas the regular potential results in the most diffuse one. For the logarithmic potential, sharpness is influenced by θ , with smaller θ yielding a sharper interface, because as θ decreases, the logarithmic potential function increasingly resembles the non-smooth obstacle potential function. These findings align with existing results in the literature, which indicate that, under suitable conditions, solutions of the nonlocal phase-field model with the obstacle potential can exhibit jump-discontinuities with only pure phases leading to a piecewise constant solution profile. This results in pure phases with no mixing during the entire time evolution (for the Cahn–Hilliard model) or at steady-state (for the Allen–Cahn model) [21,22]. In contrast, for the regular potential, interfaces in the solution, while allowing discontinuities in the solution profile, still exhibit narrow transition regions between phases due to some degree of mixing. This behavior has been studied in [23–25] for steady-state solutions of the Allen–Cahn equation. To illustrate this, in Fig. 2.1, in addition to plotting three distinct potential functions (left) and the corresponding subdifferentials (middle), the plots corresponding to steady-state solutions for $\xi = 0$, with the initial condition $u_0 = 0.1 \sin(2 * \pi x)$, are shown in Fig. 2.1 (right).

3. Discretization of the nonlocal Allen–Cahn and Cahn Hilliard equations

In this section, we introduce full spatial and temporal discretizations of the nonlocal AC and CH equations governed by (2.9) which will be used in Section 4.2 to define the loss functions. We consider first and second-order time stepping schemes combined with the Fourier collocation approach for the spatial discretization. For ease of illustration, we define the domain $\Omega = (-X, X) \times (-Y, Y) \subset \mathbb{R}^2$ and assume that u satisfies periodic boundary conditions.

3.1. Temporal discretization

We adopt first and second order time stepping schemes to discretize the model in (2.9). We choose a uniform time step size $\Delta t > 0$ and set $t_n = n\Delta t$ for $n = 0, \dots, N$, and also denote $U_n = u(x, y, t_n)$ and $U_{n/2}^{i,j} := (U_n + U_{n+1})/2$. For additional details about the study of the time-stepping schemes adopted here, we refer to [21,22,26].

¹ By a slight abuse of notation and for the sake of exposition, we use an equality in (2.9) and throughout the paper. However, this should be understood as an inclusion: $w - \xi u + \gamma_\delta * u - \partial\psi(u) \ni 0$, which reduces to the equality in the cases of the regular and logarithmic potentials.

3.1.1. Temporal discretization for the nonlocal Allen-Cahn model

For the Allen-Cahn case ($\beta = 0$) we use a first order semi-implicit discretization with an explicit discretization of the convolution term, namely, given U_0 , we seek U_{n+1} for $n \geq 0$ such that

$$\frac{1}{\Delta t}(U_{n+1} - U_n) + \xi U_{n+1} - \gamma_\delta * U_n + \partial\psi(U_{n+1}) = 0. \quad (3.1)$$

Analogously, for the second-order scheme we use for the regular or logarithmic potentials is given by

$$\frac{1}{\Delta t}(U_{n+1} - U_n) + \xi U_{n/2} - \gamma_\delta * U_{n/2} + \frac{1}{2}(\psi'(U_{n+1}) + \psi'(U_n)) = 0, \quad (3.2)$$

and for the obstacle potential in (2.8c) we use

$$\frac{1}{\Delta t}(U_{n+1} - U_n) + \xi U_{n/2} - \gamma_\delta * U_{n/2} + \partial\psi(U_{n+1}) = 0. \quad (3.3)$$

We also comment on the solution algorithms for the above systems. For the case of the regular or logarithmic potentials, the resulting systems are nonlinear and we can adopt, e.g., a fixed-point iteration to solve them. The situation is more delicate for the case of an obstacle potential, as the system is nonlinear and non-smooth. However, the structure of the problem in (2.9) allows us to derive a characterization of the solution in terms of a representation formula given by proximal operators [26]. Specifically, for example, for the first order scheme in (3.1), the solution U_n in (3.1) is equivalent to

$$U_{n+1} = \text{prox}_{\frac{1}{\lambda}\psi} \left(\frac{1}{\lambda} \left(\gamma_\delta * U_n + \frac{1}{\Delta t} U_n \right) \right), \quad \lambda = \xi + 1/\Delta t > 0, \quad (3.4)$$

and the proximal operator is defined as $\text{prox}_{\eta\psi} := (I + \eta\partial\psi)^{-1}$, $\eta > 0$ [27]. For the regular or obstacle potential the proximal operator has an analytic representation obtained by Cardano's formula [28] or by pointwise projection, respectively. That is, for the regular potential, the solution of (3.1) has the form

$$U_{n+1} = \sqrt[3]{\zeta_n + \sqrt{\zeta_n^2 + (\lambda/3c_F)^3}} + \sqrt[3]{\zeta_n - \sqrt{\zeta_n^2 + (\lambda/3c_F)^3}}, \quad (3.5)$$

where $\zeta_n = (\gamma_\delta * U_n + \frac{1}{\Delta t} U_n) / (2c_F)$ and for the obstacle potential, the proximal operator is a pointwise projection onto an admissible set and (3.4) reduces to

$$U_{n+1} = P_{[-1,1]} \left(\frac{1}{\lambda} \left(\gamma_\delta * U_n + \frac{1}{\Delta t} U_n \right) \right), \quad \lambda = \xi + 1/\Delta t > 0, \quad (3.6)$$

where $P_{[-1,1]}(\eta) = \eta$ if $|\eta| < 1$ and $P_{[-1,1]}(\eta) = \pm 1$ if $\eta \geq 1$ or $\eta \leq -1$, respectively. Analogously, for the second-order scheme the solution of Eqs. (3.2) and (3.3) has the form

$$U_{n+1} = \text{prox}_{\frac{1}{2\lambda}\psi} \left(\frac{1}{\lambda} \left((\lambda - \xi)U_n + \gamma_\delta * U_{n/2} - \frac{1}{2}\psi^*(U_n) \right) \right), \quad \lambda = \xi/2 + 1/\Delta t > 0, \quad (3.7)$$

where $\psi^*(u^{k-1}) = \psi'(u^{k-1})$ for the regular and logarithmic potentials and $\psi^*(u^{k-1}) = 0$ for the obstacle potential. Furthermore, for the obstacle potential this further reduces to

$$U_{n+1} = P_{[-1,1]} \left(\frac{1}{\lambda} \left((\lambda - \xi)U_n + \gamma_\delta * U_{n/2} \right) \right), \quad \lambda = \xi/2 + 1/\Delta t > 0. \quad (3.8)$$

Whereas the first order scheme involves only explicit evaluation of the convolution that can be performed rather efficiently, the accuracy is only of first order. On the other hand, realizing the second-order scheme in (3.2) and (3.3) or equivalently (3.7) and (3.8) requires nonlinear iterative solvers such as the Picard iteration. For example, the Picard iteration for (3.8) proceeds as follows: initialize $U_{n+1,(0)} = U_n$ and for $m = 0, 1, \dots$, update $U_{n+1,(m+1)}$ by solving

$$U_{n+1,m+1} = P_{[-1,1]} \left(\frac{1}{\lambda} \left((\lambda - \xi)U_n + \gamma_\delta * U_{n/2,m} \right) \right), \quad \lambda = \xi/2 + 1/\Delta t, \quad (3.9)$$

where $\gamma_\delta * U_{n/2,m} = \gamma_\delta (U_{n+1,m} + U_n) / 2$ and the iteration continues until $U_{n+1} = U_{N+1,m^*}$ satisfies a predefined convergence criterion.

3.1.2. Temporal discretization for the nonlocal CH equation

We briefly outline the temporal discretization for the Cahn-Hilliard model in (2.9) ($\beta > 0$). In particular, we consider a first-order semi-implicit scheme, namely, given $U_0 = u(0)$ we seek U_{n+1} , $n \geq 0$ such that

$$\begin{cases} \frac{1}{\Delta t}(U_{n+1} - U_n) + AW_{n+1} = 0, \\ W_{n+1} - \xi U_{n+1} + \gamma_\delta * U_{n-1} - \partial\psi(U_{n+1}) = 0, \end{cases} \quad (3.10)$$

or alternatively

$$-\frac{1}{\Delta t}G(U_{n+1} - U_n) + \xi U_{n+1} - \gamma_\delta * U_n + \partial\psi(U_{n+1}) = 0. \quad (3.11)$$

For the regular or logarithmic potentials, the system in (3.10) can be solved directly, e.g., using a fixed-point iteration. Alternatively, we resort to the formulation in (3.11) where we also adopt an equivalent representation of the solution through the proximal operators [26], that is, the solution of (3.11) is equivalently expressed as

$$U_{n+1} = \text{prox}_{\frac{1}{\lambda}\psi} \left(\frac{1}{\lambda} \left(\frac{1}{\Delta t} (G + cI) (U_{n+1} - U_n) + \gamma_\delta * U_n + \frac{c}{\Delta t} U_n \right) \right), \quad \lambda = \xi + \frac{c}{\Delta t} > 0, \quad (3.12)$$

where for the regular potential the proximal operator can be computed using Cardano's formula and for the obstacle potential this leads to the projection formula

$$U_{n+1} = P_{[-1,1]} \left(\frac{1}{\lambda} \left(\frac{1}{\Delta t} (G + cI) (U_{n+1} - U_n) + \gamma_\delta * U_n + \frac{c}{\Delta t} U_n \right) \right), \quad \lambda = \xi + \frac{c}{\Delta t} > 0, \quad (3.13)$$

where we have also added a stabilization term $\pm \frac{c}{\Delta t} (U_{n+1} - U_n)$, $c > 0$, in (3.11). We use the above representations to compute solutions, where, similarly as in (3.9), we perform a fixed point iteration to resolve implicit terms in (3.12).

We also comment that there exist alternative time-stepping methods including unconditionally stable schemes for the nonlocal Cahn–Hilliard or Allen–Cahn equations, see, e.g., [23,29–31], which can also be adopted in our learning framework. However, those approaches are focused solely on smooth potentials, are not applicable for the non-smooth obstacle potential.

3.2. Spatial discretization

Several numerical methods exist for spatial discretizations of the nonlocal Allen–Cahn or Cahn–Hilliard models. Mostly the schemes are designed for smooth systems involving regular or logarithmic potentials. The schemes include finite difference discretizations [30–32], spectral methods [24,25,33], finite elements [34], etc; see also [35]. An efficient method, if applicable, is to use spectral or collocation approaches for the convolution operators, where the convolution is approximated by the Fourier transform with FFT implementation. This allows for efficient evaluation of the convolution, particularly in high dimensions. Here, we adopt a Fourier collocation method as described in [26,29].

We define the discretization of Ω at collocation points which are equidistantly spaced in each dimension:

$$\Omega_h = \{(x_i, y_j) : x_i = -X + ih_x, y_j = -Y + jh_y, \quad 0 \leq i \leq N_x - 1, \quad 0 \leq j \leq N_y - 1\},$$

with uniform mesh sizes $h_x = 2X/N_x$, $h_y = 2Y/N_y$, where N_x and N_y are even numbers, and we set $h = \max\{h_x, h_y\}$. We next introduce index sets for the spatial and frequency domains:

$$\begin{aligned} S_h &= \{(i, j) \in \mathbb{Z}^2 : 0 \leq i \leq N_x - 1, \quad 0 \leq j \leq N_y - 1\}, \\ \hat{S}_h &= \left\{ (l, m) \in \mathbb{Z}^2 : -\frac{N_x}{2} + 1 \leq l \leq \frac{N_x}{2}, \quad -\frac{N_y}{2} + 1 \leq m \leq \frac{N_y}{2} \right\}. \end{aligned}$$

For all grid periodic functions $U, V : \Omega_h \rightarrow \mathbb{R}$, we define the discrete L^2 inner product $\langle \cdot, \cdot \rangle$ and norm $\|\cdot\|_2$

$$\langle U, V \rangle = h_x h_y \sum_{(i,j) \in S_h} U^{ij} V^{ij}, \quad \|U\|_2 = \sqrt{\langle U, U \rangle},$$

where $U^{ij} = u(x_i, y_j)$. By $\circledast$ we denote a discrete circular convolution for the periodic functions U, V :

$$(U \circledast V)^{ij} = h_x h_y \sum_{(p,q) \in S_h} U^{i-p, j-q} V^{pq}, \quad (i, j) \in S_h.$$

We employ a discrete Fourier transform using FFT to efficiently compute the discrete convolution with $\mathcal{O}(N \log N)$ complexity. To this end, we define a discrete Fourier transform

$$\hat{U}^{lm} = (\mathcal{F}_N U)^{lm} = \sum_{(i,j) \in S_h} U^{ij} e^{-i\pi \left(\frac{lx_i}{X} + \frac{my_j}{Y} \right)}, \quad (l, m) \in \hat{S}_h. \quad (3.14)$$

and the corresponding inverse discrete Fourier transform

$$U^{ij} = \left(\mathcal{F}_N^{-1} \hat{U} \right)^{ij} = \frac{1}{N_x N_y} \sum_{(l,m) \in \hat{S}_h} \hat{U}^{lm} e^{i\pi \left(\frac{lx_i}{X} + \frac{my_j}{Y} \right)}, \quad (i, j) \in S_h. \quad (3.15)$$

Then, taking into account that

$$(\widehat{U \circledast V})^{lm} = h_x h_y \hat{U}^{lm} \hat{V}^{lm}, \quad (l, m) \in \hat{S}_h,$$

we have that

$$(U \circledast V)^{ij} = \left(\mathcal{F}_N^{-1} (\widehat{U \circledast V}) \right)^{ij} = \frac{h_x h_y}{N_x N_y} \sum_{(l,m) \in \hat{S}_h} \hat{U}^{lm} \hat{V}^{lm} e^{i\pi \left(\frac{lx_i}{X} + \frac{my_j}{Y} \right)}, \quad (i, j) \in S_h.$$

Now, by taking $U \equiv 1$ and $V = \gamma_\delta^N$ in the above we can directly compute $\xi_N = c_{\gamma_\delta}^N = c_F^N$ with

$$c_{\gamma_\delta}^N = \gamma_\delta^N \circledast 1 = h_x h_y \widehat{\gamma_\delta^N} = h_x h_y \sum_{(i,j) \in S_h} \gamma_\delta^{N,ij}.$$

Then, for example, the fully discrete system for (3.1) becomes:

$$\frac{1}{\Delta t} (U_{n+1}^{i,j} - U_n^{i,j}) + \xi_N U_{n+1}^{i,j} - \gamma_\delta * U_n^{i,j} + \partial \psi(U_{n+1}^{i,j}) = 0, \quad (i, j) \in S_h,$$

where the solution also admits a discrete version of a representation formula (3.4):

$$U_{n+1}^{i,j} = \text{prox}_{\frac{1}{\lambda_N} \psi} \left(\frac{1}{\lambda_N} \left(\gamma_\delta \circledast U_n^{i,j} + \frac{1}{\Delta t} U_n^{i,j} \right) \right), \quad \lambda_N = \xi_N + 1/\Delta t, \quad (i, j) \in S_h.$$

Analogously, we can derive a fully discrete system having second-order accuracy and a fully discrete system for the Cahn–Hilliard model.

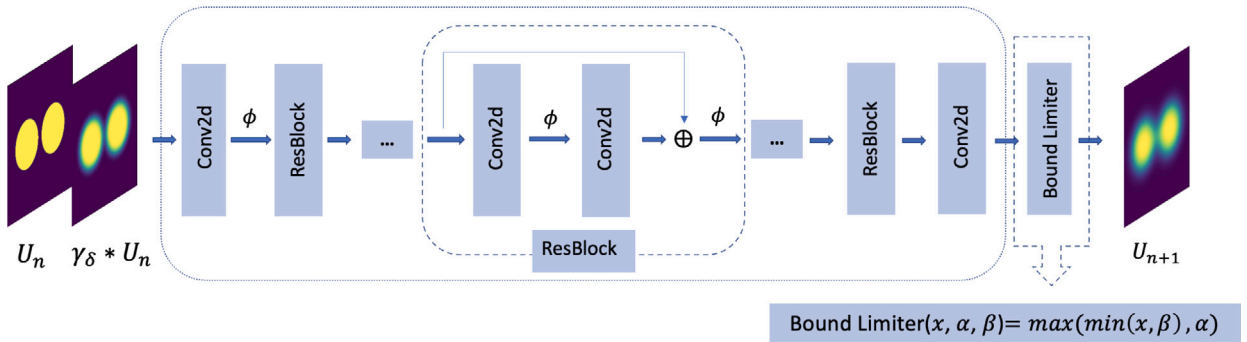


Fig. 4.1. The network architecture of NPF-Net which learns the dynamics of non-mass conserving nonlocal phase field models (2.1). In this work, it is used to approximate the operator $\mathcal{H}_*$ with $U_{n+1}^{i,j,(l)} = \text{NPF-Net}(U_n^{i,j,(l)}; \Theta) \approx \mathcal{H}_*(U_n^{i,j,(l)})$, mapping U_n to U_{n+1} , where Θ denotes the set of learnable parameters in NPF-Net.

4. The nonlocal phase-field network (NPF-Net)

We now introduce a deep learning method aimed at learning the fully discrete operators $\mathcal{H}_*$, which map the input U_n to the output U_{n+1} , without requiring ground-truth data for training. Specifically, we define $\mathcal{H}_*$, where $*$ $\in \{AC, CH\}$ as the operator given in Section 3. The objective is to accurately approximate these operators, thereby enabling rapid prediction of the dynamics of the nonlocal AC and CH equations under arbitrary initial conditions, without the necessity for retraining.

4.1. Network architecture

Motivated by the network model in [19], we propose a network architecture to model the nonlinear mappings $\mathcal{H}_*$ from U_n to U_{n+1} with the fixed time step size Δt . Fig. 4.1 illustrates the proposed “NPF-Net” architecture designed to learn the dynamics of the nonlocal phase-field models described by (2.1). This architecture includes a single convolutional layer, one or more successive Residual blocks (Resblocks) [36], and a final layer. The input to the network consists of U_n and $\gamma_\delta * U_n$, represented as $1 \times N_x \times N_y$ tensors. These are combined into a $2 \times N_x \times N_y$ tensor which serves as the input to predict the solution at the next time step, U_{n+1} , using a loss function based on the fully discrete scheme derived in Section 3. The filter size in each convolutional layer is fixed as $K \times K$, and the number of filters C is consistent across all intermediate layers. The first layer broadcasts the input tensor to the shape of $C \times N_x \times N_y$, whereas the last layer reduces it back to $1 \times N_x \times N_y$. Given the periodic boundary conditions of the target problem, the “circular” padding [37] model is applied across all convolutional layers. To preserve the spatial dimensions of the input feature map, the padding size is set according to the filter size used in each layer. ResBlocks are incorporated into the NPF-Net architecture to address the issue of vanishing gradients and improve gradient flow during training. Each ResBlock consists of two convolutional layers followed by an activation function ϕ . It is important to note that violating the Maximum Bound Principle (MBP) may result in overflow errors when evaluating the loss function in the logarithmic potential and obstacle potential cases due to the presence of logarithmic terms and non-smoothness, respectively, leading to training failure. To uphold the MBP, we incorporate a bound limiter module [19] into the network architecture which ensures that the network output remains within the interval $[\alpha, \beta]$.

4.2. Loss function

To learn the nonlinear mapping $\mathcal{H}_*$ by the discrete system, we define the loss function based on the residual of the underlying discretized system discussed in Section 3. Let the network input be denoted by $U_n = \{U_n^{(1)}, U_n^{(2)}, \dots, U_n^{(S)}\}$, where $U_n^{(l)}$ represents the solution at the time t_n corresponding to the l th problem. The loss function with the regular and logarithm potential function is formulated to measure the violation (3.2):

$$\mathcal{L}_{\text{NPF-Net}} = \frac{1}{SN^2} \sum_{l=1}^S \sum_{i,j=0}^{N-1} \left[U_{n+1}^{i,j,(l)} - U_n^{i,j,(l)} + \Delta t \left(\xi_N U_{n/2}^{i,j,(l)} - \gamma_\delta \otimes U_{n/2}^{i,j,(l)} + \left(\psi'(U_n^{i,j,(l)}) + \psi'(U_{n+1}^{i,j,(l)}) \right) / 2 \right) \right]^2, \quad (4.1)$$

where we recall $U_{n/2}^{i,j,(l)} = (U_{n+1}^{i,j,(l)} + U_n^{i,j,(l)}) / 2$. For the obstacle potential, to deal with the non-smooth function $\partial\psi_{obs}$, we define loss functions to measure the violation (3.8) with projection operator:

$$\mathcal{L}_{\text{NPF-Net}} = \frac{1}{SN^2} \sum_{l=1}^S \sum_{i,j=0}^{N-1} \left[U_{n+1}^{i,j,(l)} - P_{[-1,1]} \left(\frac{1}{\frac{1}{\Delta t} + \frac{\xi_N}{2}} \left[\left(\frac{1}{\Delta t} - \frac{\xi_N}{2} \right) U_n^{i,j,(l)} + \gamma_\delta \otimes U_{n/2}^{i,j,(l)} \right] \right) \right]^2. \quad (4.2)$$

Thus, $U_{n+1}^{i,j,(l)} = \text{NPF-Net}(U_n^{i,j,(l)}; \Theta) \approx \mathcal{H}_{AC}(U_n^{i,j,(l)})$, where $\{U_{n+1}^{i,j,(l)}\}$ denotes the approximate solution of the l th problem. Note that adding the bound limiter can help us learn the nonlinear operator with a non-smooth potential function with $\{U_{n+1}^{i,j,(l)}\}_{i,j=1}^N \in [-1, 1]$ by the bound limiter.

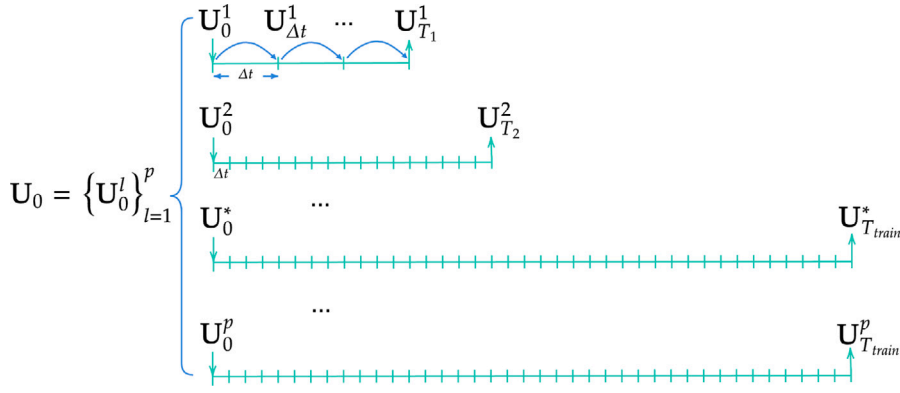


Fig. 4.2. Illustration of the time adaptive training strategy, where training begins with a short time interval $[0, T_1]$ and gradually expands as the training progresses until T_{train} .

Similarly, for the CH equation with a smooth potential function, the loss function can be defined as

$$\mathcal{L}_{\text{NPF-Net}} = \frac{1}{SN^2} \sum_{l=1}^S \sum_{i,j=0}^{N-1} \left[U_{n+1}^{i,j,(l)} - U_n^{i,j,(l)} + \Delta t A \left(\xi_N(U_{n+1}^{i,j,(l)}) - \gamma_\delta \odot (U_n^{i,j,(l)}) + \psi'(U_{n+1}^{i,j,(l)}) \right) \right]^2, \quad (4.3)$$

where $A = I - \beta \Delta$, $\beta > 0$, and for the obstacle potential function, the loss function is defined by

$$\mathcal{L}_{\text{NPF-Net}} = \frac{1}{SN^2} \sum_{l=1}^S \sum_{i,j=0}^{N-1} \left[U_{n+1}^{i,j,(l)} - P_{[-1,1]} \left(\frac{1}{\frac{c}{\Delta t} + \xi_N} \left[\frac{1}{\Delta t} (G + cI) (U_{n+1}^{i,j,(l)} - U_n^{i,j,(l)}) + \gamma_\delta \odot U_n^{i,j,(l)} + \frac{c}{\Delta t} U_n^{i,j,(l)} \right] \right) \right]^2, \quad (4.4)$$

where $U_{n+1}^{i,j,(l)} = \text{NPF-Net}(U_n^{i,j,(l)}; \Theta) \approx \mathcal{H}_{CH}(U_n^{i,j,(l)})$.

It is important to note that, due to the specific selection of loss functions, NPF-Net can be trained without needing any ground truth data. This characteristic distinguishes these models, allowing them to learn effectively from the underlying dynamics of the problem rather than relying on pre-existing solution data.

4.3. Time-adaptive training strategy

Next, we describe the adaptive training strategy for NPF-Net. Consider a scenario with S problems, each having different initial conditions, where S can be expressed as $S = pq$. The initial data for these problems is organized into $U_0 = \{U_0^{(l)}\}_{l=1}^S$, with each value $\{U_0^{i,j,(l)}\}_{i,j=1}^N$ constrained within the interval $[-1, 1]$. The training is conducted over the time interval $[0, T_{train}]$, where $T_{train} = a\Delta t$ and a is a positive integer. Recall the training strategy in [19]; there, with fixed training initial conditions, the model was trained sequentially from initial conditions to T_{train} for each subset in U_0 . In this paper, we introduced adaptive strategies in time to reduce the training time, meanwhile improving the accuracy. Fig. 4.2 illustrates the detailed training with fixed Δt .

- (i). The initial set $U_0 = \{U_0^{(1)}, U_0^{(2)}, \dots, U_0^{(S)}\}$ is randomly partitioned into p subsets, denoted as $\{U_0^l\}_{l=1}^p$, with each subset containing q distinct initial conditions such that $S = pq$.
- (ii). The neural network is initially trained using the first subset U_0^1 to learn the mapping from U_0^1 to $U_{T_1}^1$ sequentially with a specified learning rate, where $T_1 \leq T_{train}$.
- (iii). Subsequently, the neural network is provided with the second subset U_0^2 and the learning rate decay by some constant factor, and the training process continues from U_0^2 to $U_{T_2}^2$, ensuring that $T_1 \leq T_2 \leq T_{train}$. This process is iterated for each subset until the designated training time T_{train} is achieved. If $T_1 = T_2 = \dots = T_{train}$, this corresponds to the training strategy outlined in [19].
- (iv). Upon reaching T_{train} with initial condition U_0^* , the model undergoes further training to learn the process from U_0 to $U_{T_{train}}$. Note, once the training time reaches T_{train} , it remains fixed, and training continues until the loss reaches an acceptable threshold or the maximum number of epochs is completed.

This approach effectively balances training time and accuracy, preventing the adverse effects of prolonged sequential training.

5. Numerical results

In this section, we conduct some hyperparameter optimizations on different network architectures and then test the trained models on benchmark problems for the nonlocal AC and CH equations. Throughout all the experiments, we fix the spatial domain $\Omega = [-1, 1]^d$, $d = 2, 3$ and use $\Delta t = 0.1$ for the nonlocal AC equation and $\Delta t = 0.01$ for the nonlocal CH equation. For the mesh grid,

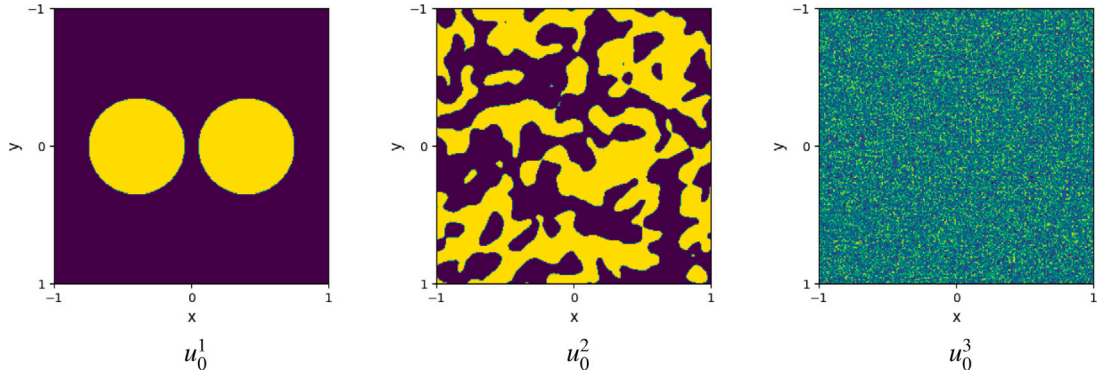


Fig. 5.1. Three different types of initial conditions. u_0^1 is for the bubbles merging case, u_0^2 is for sharp-colored noise, and u_0^3 is for white noise initial condition.

we fix $N_x = N_y = 256$ for two-dimensional case ($d = 2$) and $N_x = N_y = N_z = 64$ for three-dimensional case ($d = 3$). We set $c_F = 1$ for all three types of potential functions and for the logarithm potential function we set $\theta_c = 0.5$. The bounds for the AC equation are as follows: for the regular and obstacle potential functions, the bounding interval is $[-1, 1]$. In the case of the logarithmic potential, the bounds are $[-\rho_1, \rho_1]$ where $\rho_1 = 0.95750402$. For the nonlocal CH equation, there are no bounds for the regular potential function, whereas for the obstacle potential function, the bound is $[-1, 1]$. We employ a smooth kernel function defined as the Gauss-type functions having the form

$$\gamma_\delta(\mathbf{x}) = \frac{4\epsilon^2}{\pi^{(d/2)}\delta^{(d+2)}} e^{-|\mathbf{x}|^2/\delta^2}, \quad \mathbf{x} \in \mathbb{R}^d, \delta > 0,$$

which is appropriately scaled to match $Bu \approx -\epsilon^2 \Delta u$ for $\delta \rightarrow 0$, and we fix $\epsilon = 0.05$. For the experiments, we select $\delta = 0.05, 0.075, 0.095, 0.1$ with corresponding values of $\xi = 3, 0.778, 0.108, 0$ respectively.

To train our networks, we follow the same parameter settings as outlined in [19] and the random initial conditions $u_0(x, y) = 0.95 \mathbf{rand}(x, y)$, where $\mathbf{rand}(x, y)$ denotes the pseudo-random number generator producing a scalar value between -1 and 1 . Unless stated otherwise, we adopt the Adam optimizer [38] with an initial learning rate of 0.001 which decays by a factor of 0.6 after the training of each subset in $\mathbf{U}_0$. Additionally, we set the maximum number of epochs to 2 and the loss tolerance to $1e-10$. Our models are implemented in PyTorch [39], and all experiments are conducted on a server equipped with a V100 GPU card with 32 GB of memory. The training time for each model in the experiments presented in this section range over 9 h for two-dimensional problems and 50 h for three-dimensional problems.

To evaluate the accuracy of our trained models, we calculate the prediction errors using the relative L_2 -norm

$$Error = \frac{\|\mathbf{U}_{\mathcal{NN}} - \mathbf{U}_{ref}\|_2}{\|\mathbf{U}_{ref}\|_2},$$

where $\mathbf{U}_{\mathcal{NN}}$ is the predicted solution generated by the trained neural network models and $\mathbf{U}_{ref}$ is the reference solution produced by the corresponding fully-discrete schemes. The reference solution is computed with the same spatial mesh and the sufficiently small time step size $\Delta t = 0.001$ for AC and $\Delta t = 1e-5$ for CH. The resulting nonlinear systems are solved using a Picard iteration with FFT-based implementation at each time step as outlined in Section 3. We consider three different types of initial conditions to test our network for solving two-dimensional nonlocal problems. The performance of these different initial conditions is shown in Fig. 5.1. For the bubbles-merging initial condition u_0^1 we define

$$u_0^1(x, y) = \begin{cases} 1, & \text{if } (x - 0.4)^2 + y^2 \leq 0.35^2 \text{ and } (x + 0.4)^2 + y^2 \leq 0.35^2, \\ -1, & \text{else.} \end{cases}$$

For the sharp-colored noise initial condition u_0^2 , we first generate the smooth Gaussian noise $G(x, y)$ by using the Python tools GSTools [40] and then we generate the sharp-colored noise initial condition u_0^2 , where the values are restricted to either 1 or -1 , as follows:

$$u_0^2(x, y) = \begin{cases} 1, & \text{if } G(x, y) \geq 0, \\ -1, & \text{if } G(x, y) < 0. \end{cases}$$

Finally, the white noise initial condition u_0^3 is selected from $u_0^3(x, y) = 0.95 \mathbf{rand}(x, y)$. To assess the generalizing ability of our network, we calculate the average L_2 error across 50 different initial conditions for both white and sharp-colored noise, as both types of initial conditions can be generated randomly.

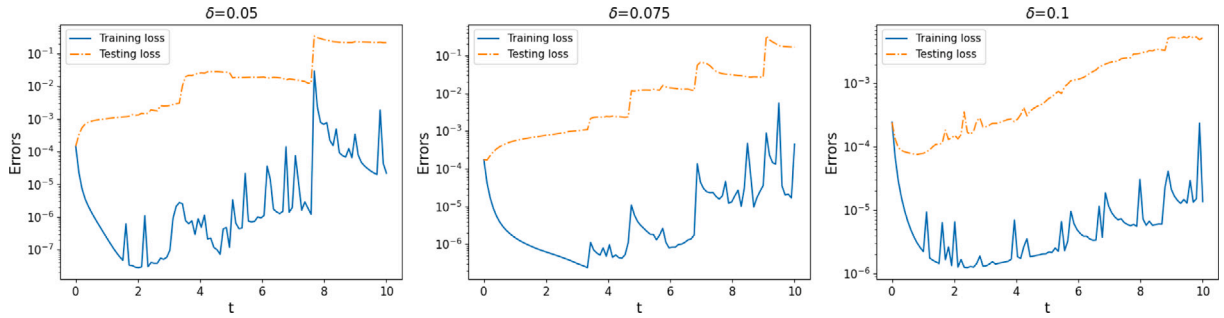


Fig. 5.2. Training loss and testing loss for training the steps from U_0 to $U_{T_{train}}$ sequentially with different δ . The testing set is 10 random initial conditions outside the training set. For all choices of δ , both training loss and test loss increase sharply, especially after $t = 2$ (20 steps).

5.1. Hyperparameter optimization

In this section, we perform hyperparameter optimizations to evaluate how improved training strategies and network architecture affect a model's prediction performance. For this analysis, we focus solely on the two-dimensional nonlocal AC equations.

Effect of the training strategy. Our first objective is to investigate how the improved training strategy affects the performance of the proposed methods for solving the nonlocal AC problem. First, we follow the sequential training strategy with $T_1 = T_{train} = 10$ on a training subset U_0^1 which contains $q = 5$ random initial conditions. Then, we test our model on 10 random initial conditions outside of the training set. We observe that as the model progressed to later time steps for the nonlocal AC equation, the training loss and testing loss increase significantly, especially after $t = 2$, as shown in Fig. 5.2. Therefore, we propose a time-adapted training strategy aimed at reducing training time while maintaining accuracy. Instead of training the model directly to T_{train} , we select $T_1 = 2$, $T_2 = 4$ until to $T_{train} = 10$ by adding $\Delta T = 2$ for each training subset. In the next section, we compare the accuracy of the sequential training method and time-adaptive training strategies on different network architectures.

Effect of the network architecture. To illustrate the advantage of employing $\gamma_\delta * U_n$ in NPF-Net, we compare the performance of ACNN to NPF-Net. Fig. 5.3 presents the average L_2 error over 50 white noise initial conditions for the sequential training strategy (top) and time-adaptive training process (bottom), respectively. For the ACNN model with the input containing one channel, we first used 3 resblocks 16 mid-channels and a fixed filter size 3 for all layers, corresponding to $C = 1$ and $K = 3$ in Fig. 5.3 (left). From the results, we observe that the errors are significantly large, especially for a large choice of δ with a long interaction domain. To solve that problem, we then consider increasing the input filter size corresponding to the different options of δ . For instance, with a mesh size 256×256 over the domain $[-1, 1]^2$ ($h = 2/256$) and $\delta = 0.1$, we select the filter size to be 27 and a padding number of 13, corresponding to $\frac{\delta}{h} = 12.8$. Similarly, the input filter size can be determined based on the choice of δ in Table 1. The results, shown in Fig. 5.3 (middle), indicate a significant error reduction with increased receptive fields by increasing the filter size. However, for large δ , the errors, although reduced, remain notable. Additionally, small mesh sizes h necessitate adjustments in filter sizes to enhance receptive fields, which consequently increases the model complexity and the risk of overfitting. To address these challenges, we propose NPF-Net for the nonlocal problem which enhances performance without increasing the receptive fields by adding the convolutional kernel $\gamma_\delta * U_n$ to the input channel with $C = 2$ and $K = 3$. Fig. 5.3 (right) demonstrates that accuracy is significantly improved for large δ values and that the accuracy remained consistent across different δ values. In addition, the time-adaptive training strategy is more accurate and stable than the original training method for different network architectures. Furthermore, while the sequential training took 13 h, the adaptive training strategy reduced the training time to 9 h (a reduction of approximately 30%). This time reduction is even more pronounced for larger-scale problems. Therefore, we use NPF-Net with the adaptive training strategy in all subsequent experiments.

Effect of different training sets. To evaluate the network's performance across different types of initial conditions, we first tested our NPF-Net model training with U_0 , which contains 20 random initial conditions (u_0^3) only. Following this, we simulated solutions with various initial conditions. The top rows in Figs. 5.4, 5.5, and 5.6 shows the error for the nonlocal AC equation with regular, logarithmic, and obstacle potential functions, respectively. We observe that the model exhibited significant errors, especially when handling sharp profiles with logarithmic and obstacle potential functions, e.g., bubbles merging u_0^1 and sharp-colored noise u_0^2 initial conditions. To address this issue, we incorporated sharp-colored noise u_0^2 into U_0 alongside the random initial conditions. For training, we selected a $U_0 = \{U_0^l\}_{l=1}^p$ containing 20 white noise initial conditions (u_0^3) and 20 sharp-colored noise conditions (u_0^2). For each training subset, we randomly selected U_0^l containing 4 white noise initial conditions and 4 sharp-colored noise initial conditions. As the bottom row in these figures shows, this approach effectively halved the error across various initial conditions. Thus, integrating sharp Gaussian noise into U_0 significantly enhances the model's generalizing capabilities and accuracy, particularly for handling sharp interfaces. Consequently, we include both white and sharp-colored initial conditions in U_0 for all subsequent experiments.

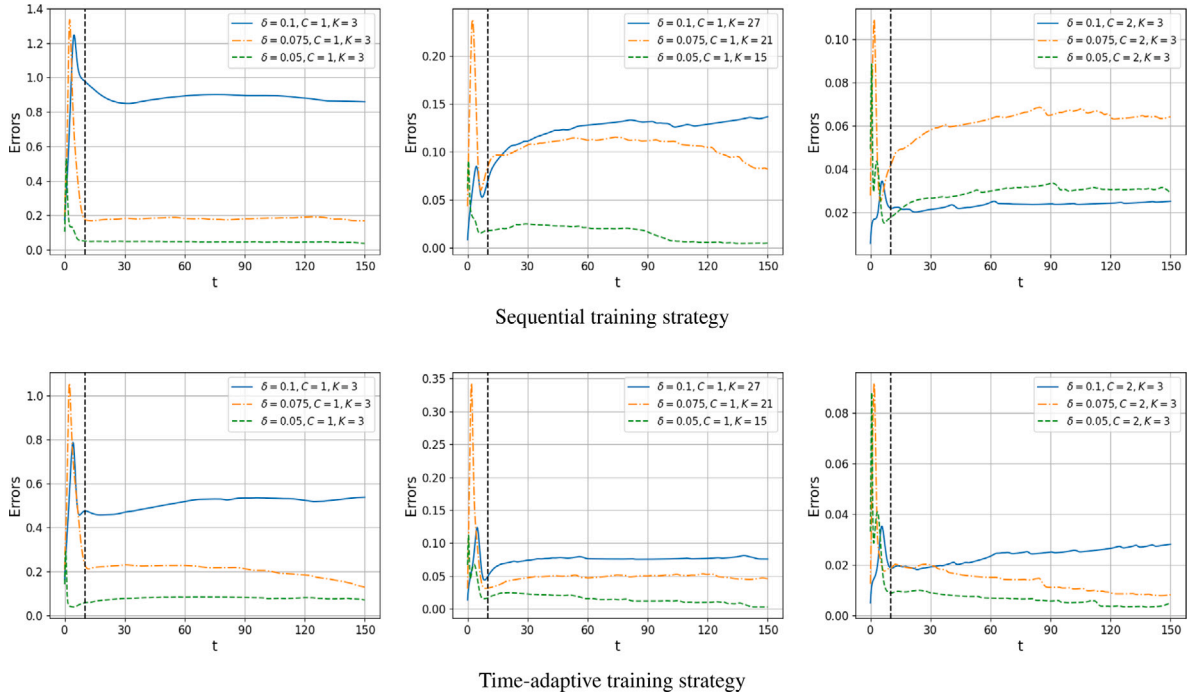


Fig. 5.3. Evolution of the model prediction errors for the regular potential case under different training strategies. A network using U_n as input with a fixed kernel size performs poorly, but the accuracy improves as the filter size K of the input layer increases. Ultimately, the network using both U_n and $\gamma_\delta * U_n$ as inputs achieves the best performance, with stable simulation errors across different choices of δ . In addition, for all network settings, the simulation errors under the time-adaptive training strategy are significantly lower than those using the sequential training method. The vertical black dashed lines indicate that the end of the training time is $T_{train} = 10$.

Table 1

The convolutional kernel size and padding correspond to different choices of δ .

Kernel, padding	$\delta = 0.05$	$\delta = 0.075$	$\delta = 0.1$
$N = 256$	15,7	21,10	27,13

5.2. Two-dimensional bubbles merging

Next, we demonstrate, through bubble-merging examples, the excellent performance of our proposed methods, both in terms of accuracy and efficiency. These examples illustrate how well the NPF-Net model learns the dynamics of the AC and CH equations. To evaluate the accuracy of our model, we compare the results using NPF-Net with the numerical scheme discussed in Section 3 using the same mesh size $N_x = N_y$ and time step size Δt . The benchmark solutions are generated by solving the numerical scheme with a finer time step and double the mesh grid resolution. In this aspect of the study, we focus solely on the bubble-merging case because doubling the mesh grid for cases with random initial conditions would lead to different solution behaviors. Additionally, we observe that solving the nonlocal phase-field model with a regular potential behaves similarly to that using a logarithmic potential. Therefore, for simplicity, in the subsequent sections we focus exclusively on systems with regular and obstacle potential functions.

5.2.1. Nonlocal Allen-Cahn equation

For the nonlocal AC equation, the NPF-Net model is trained with $N_x = N_y = 256$ and $\Delta t = 0.1$. For $N_x = N_y = 512$ and $\Delta t = 0.001$, to test for accuracy, NPF-Net solutions are compared to the approximate solutions obtained using the numerical schemes discussed in Section 3. Fig. 5.7 illustrates the errors when solving the nonlocal AC equation with the NPF-Net model and the Crank-Nicholson scheme. We observe that, for $\delta = 0.05$ and 0.075 , the prediction errors from NPF-Net are similar to those generated by the Crank-Nicholson scheme throughout the simulation. However, for $\delta = 0.1$ the prediction errors from NPF-Net are four times larger than those from the numerical simulations. To show the efficiency of NPF-Net, in Table 2, we show the GPU times for simulating solutions until $T = 100$ with NPF-Net and a numerical scheme. NPF-Net shows a clear advantage in execution time, remaining largely unaffected by changes in δ or the type of potential function used. This consistency makes NPF-Net a more efficient and robust solution. Fig. 5.8 presents the predicted solution and associated errors at the times $t = 10, 20, 50, 100$ for the regular and obstacle potential functions. From top to bottom, the results are for solving the discretized nonlocal AC equations for $\delta = 0.05, 0.075, 0.1$.

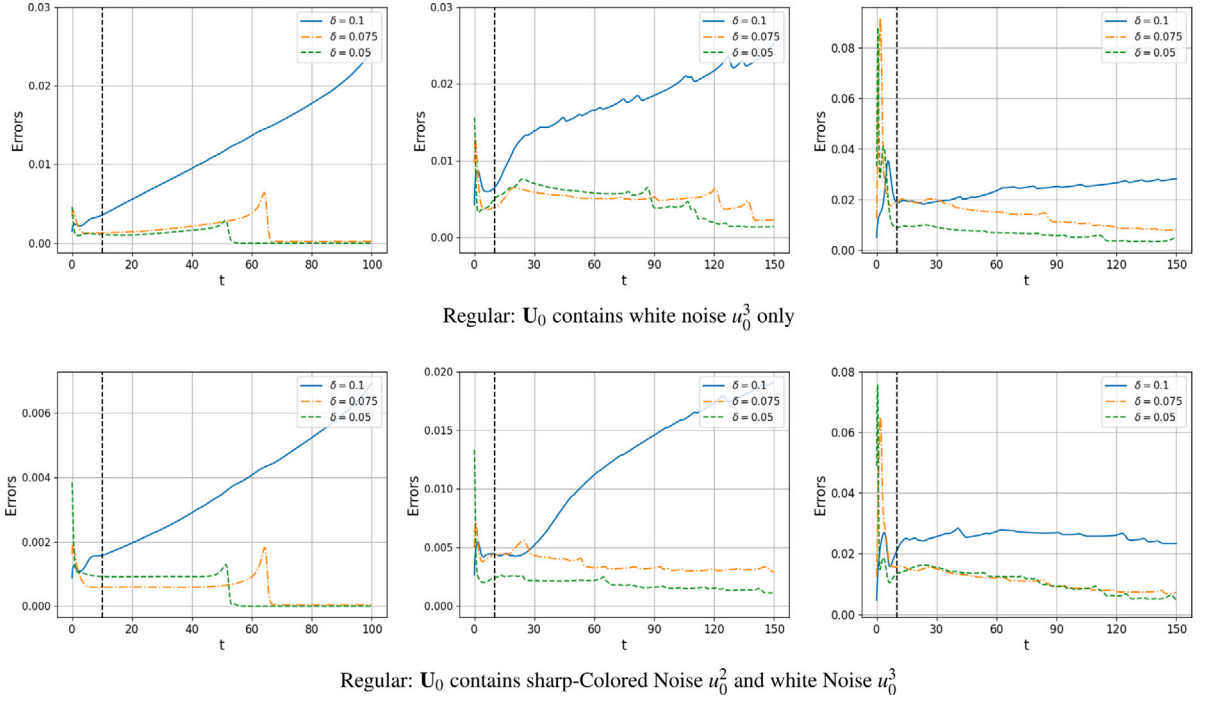


Fig. 5.4. Evolution of model prediction errors for the nonlocal AC equation with the regular potential function, tested with different initial conditions: u_0^1 (left), u_0^2 (middle), and u_0^3 (right). Top row: the NPH-Net model is trained using an initial training set U_0 containing only u_0^3 . Bottom row: the network model is trained with the initial training set U_0 incorporated with sharp-colored noise u_0^2 . Comparing the two training choices, it is evident that incorporating sharp-colored noise into the initial training set significantly improves the model's accuracy across different initial conditions. The vertical black dashed line indicates that the end of the training time is $T_{train} = 10$.

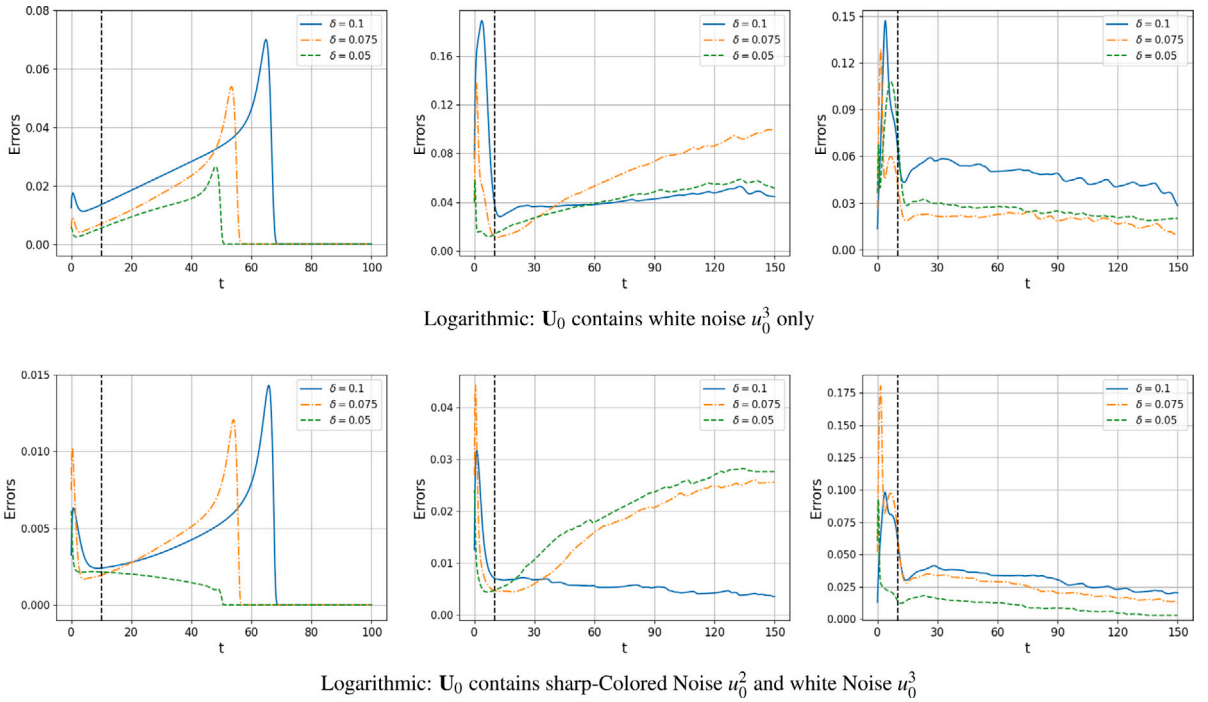


Fig. 5.5. Evolution of model prediction errors for the nonlocal AC equation with the logarithmic potential function, tested with different initial conditions: u_0^1 (left), u_0^2 (middle), and u_0^3 (right). The vertical black dashed line indicates that the end of the training time is $T_{train} = 10$.

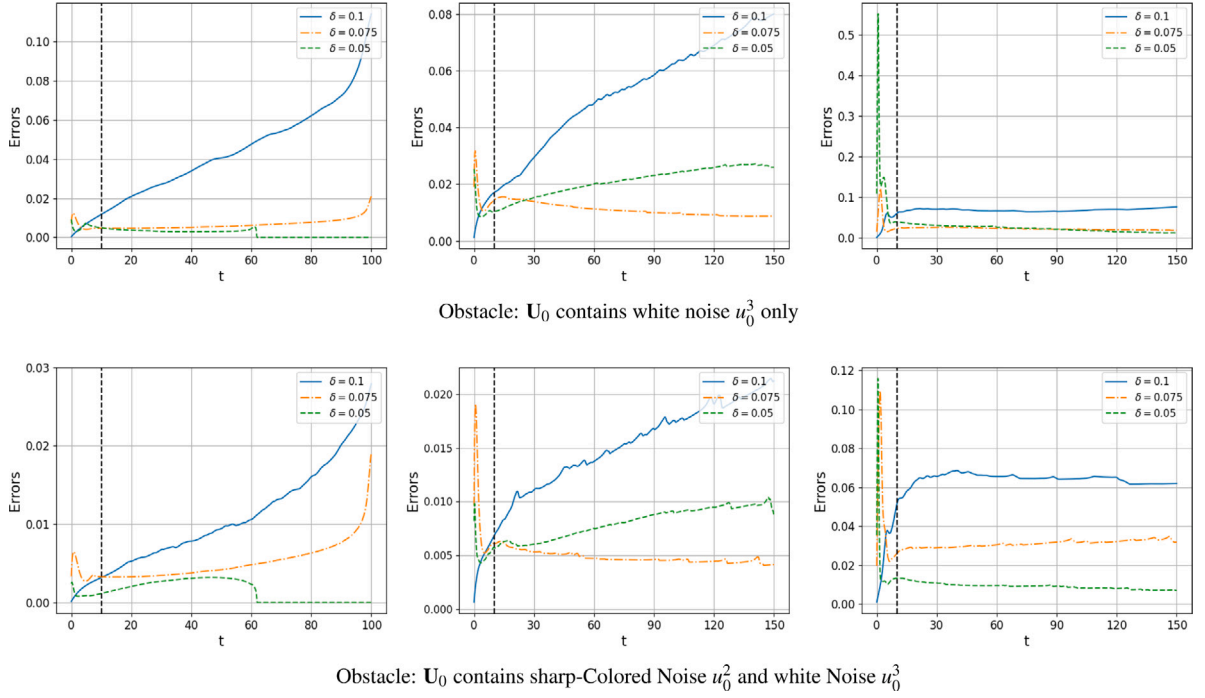


Fig. 5.6. Evolution of model prediction errors for the nonlocal AC equation with the obstacle potential function, tested with different initial conditions: u_0^1 (left), u_0^2 (middle), and u_0^3 (right). The vertical black dashed line indicates that the end of the training time is $T_{train} = 10$.

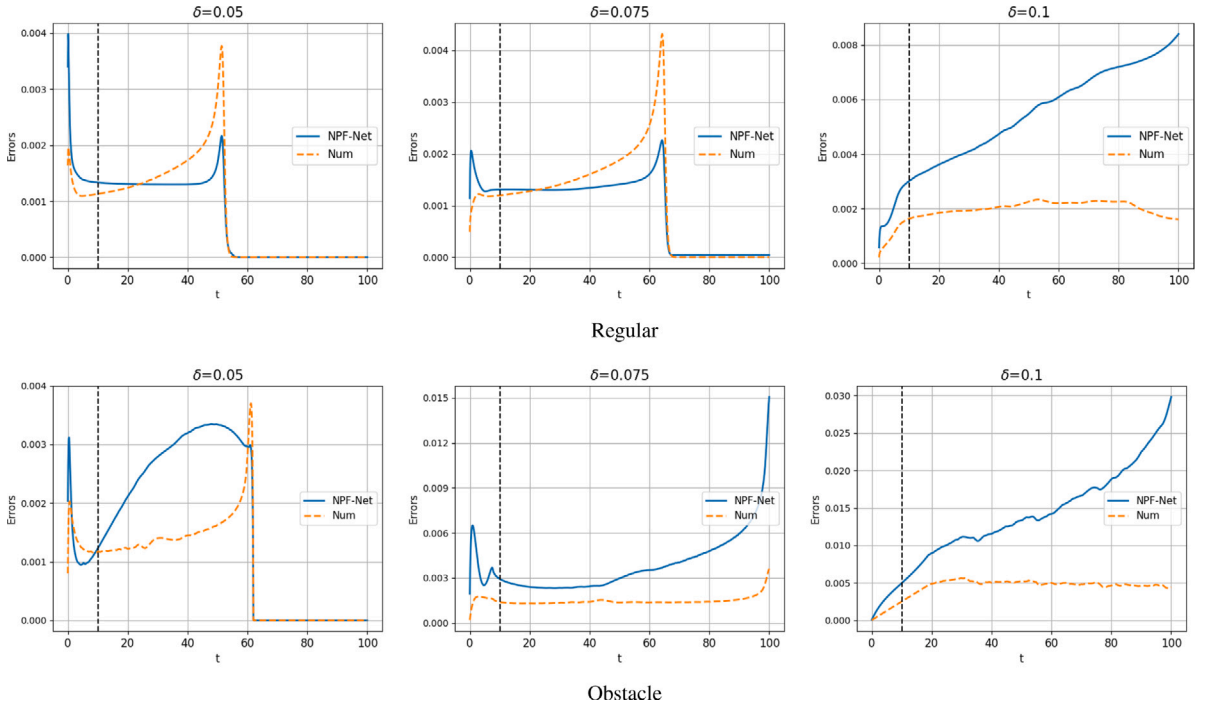


Fig. 5.7. Evolution of model prediction errors for the nonlocal AC equation when using the NPF-Net model and the Crank–Nicolson scheme with $N_x = N_y = 256$, $\Delta t = 0.1$, and for different choices of δ . The vertical black dashed line indicates that the end of the training time is $T_{train} = 10$.

Table 2For the Allen–Cahn equation, a comparison of GPU times (in seconds) for different δ values.

GPU time (s)		$\delta = 0.05$	$\delta = 0.075$	$\delta = 0.1$
Regular	NPF-Net	1.546	1.584	1.553
	Numerical scheme	1.493	1.409	1.882
Obstacle	NPF-Net	1.606	1.537	1.607
	Numerical scheme	2.084	2.290	1.490

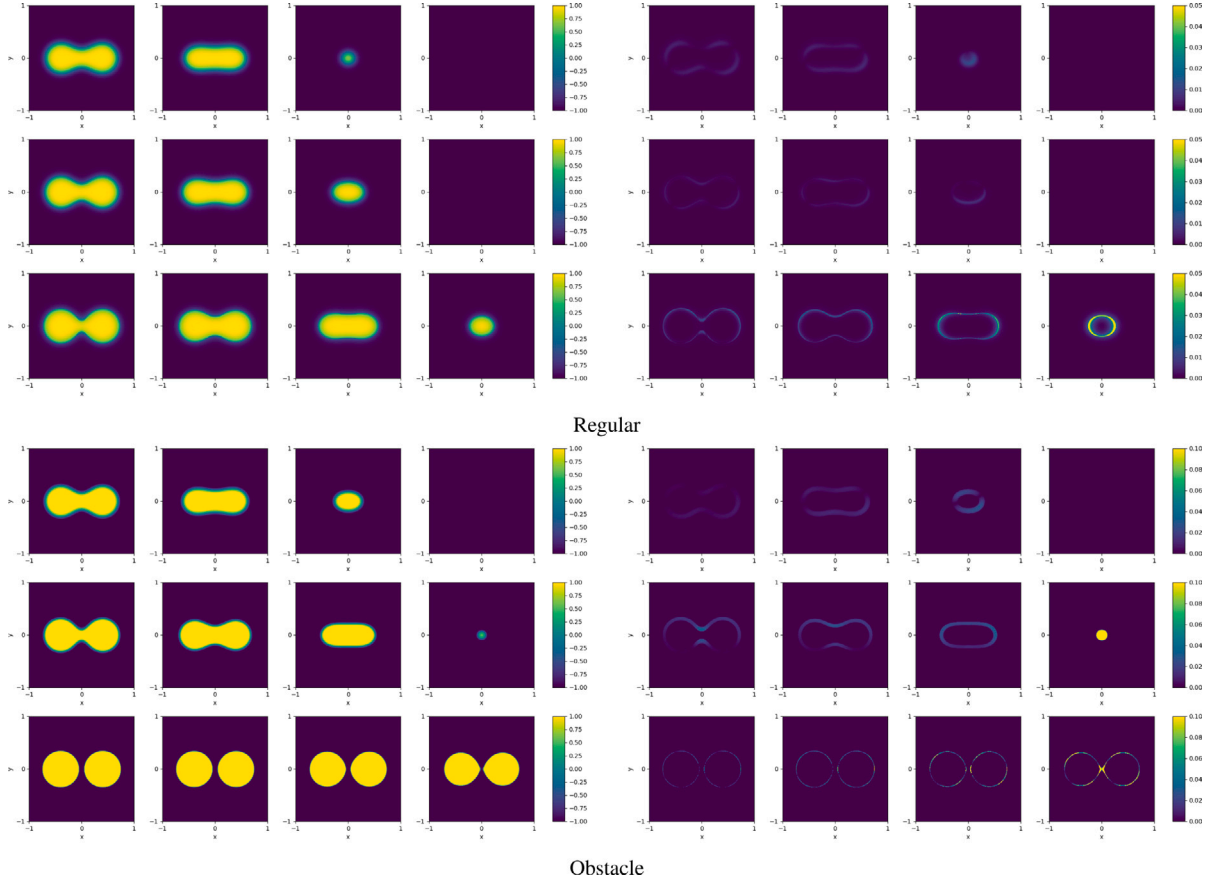


Fig. 5.8. For two-dimensional merging bubbles example, plots of the NPF-Net prediction solutions (left columns) and the corresponding numerical errors (right columns) for solving the nonlocal AC equation with the regular and obstacle potentials. The rows, from top to bottom, correspond to results at the different values of $\delta = 0.05, 0.075, 0.1$. The columns, from left to right, display the results at four different times $t = 10, 20, 50, 100$.

We observe that the two separate bubbles gradually shrink and merge into a smaller bubble which keeps shrinking and finally disappears at different times for the different choices of δ . The prediction errors seem to only occur in the transition regions of two phases, which implies that phase shapes are accurately captured.

5.2.2. Nonlocal Cahn–Hilliard equation

For the nonlocal CH equation, the NPF-Net model was trained with $N_x = N_y = 256$, for $T_{train} = 2$ with $\Delta t = 0.01$. For $N_x = N_y = 512$ and $\Delta t = 1e-5$, to test for accuracy, NPF-Net solutions are compared to the approximate solutions obtained using the numerical schemes discussed in Section 3. Fig. 5.9 illustrates the comparison of errors of NPF-Net and the numerical solver for different choices of δ . We observe that the prediction errors for NPF-Net are comparable with that of the numerical solver. In Table 3, the GPU times for simulating the solutions up to $T = 5$ are presented for both the numerical scheme and NPF-Net. The GPU time for the numerical scheme increases with larger δ due to the additional iterations required. However, the GPU times for NPF-Net remain stable, unaffected by changes in δ or the choice of potential function. Overall, NPF-Net is significantly more efficient in simulating the solution compared to the numerical scheme based on iterative methods. It is worth noting that the iterative method

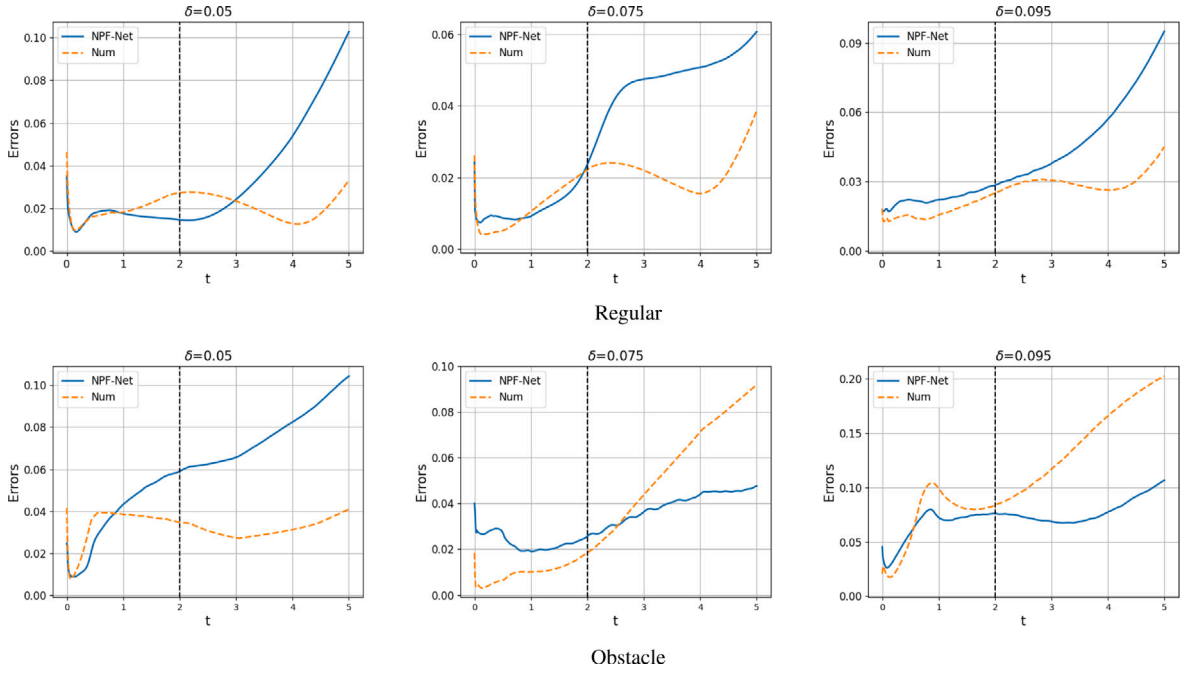


Fig. 5.9. Errors network simulations and numerical prediction with $N_x = N_y = 256$ and $\Delta t = 0.01$ for different choice of $\delta = 0.05, 0.075, 0.095$. The vertical black dashed line indicates the end of the training time is $T_{train} = 2$.

Table 3

For the Cahn–Hilliard equation, a comparison of GPU times (in seconds) for different δ values.

GPU time (s)		$\delta = 0.05$	$\delta = 0.075$	$\delta = 0.095$
Regular	NPF-Net	0.816	0.782	0.763
	Numerical scheme	20.512	62.472	422.592
Obstacle	NPF-Net	0.883	0.848	0.812
	Numerical scheme	15.647	46.877	242.920

converges faster when solving the nonlocal AC equation compared to the nonlocal CH equation, resulting in lower GPU times for solving nonlocal AC than for nonlocal CH. Fig. 5.10 presents the predicted solution and associated errors at the times $t = 0.5, 1, 2, 5$ for the regular and obstacle potential functions. From top to bottom, the results are for solving the nonlocal CH equations for $\delta = 0.05, 0.075, 0.095$. We observe that the two separate bubbles gradually merge into one bubble.

5.3. Two-dimensional grain coarsening

Next, we use the trained NPF-Net to simulate the two-dimensional grain coarsening problem for which the initial conditions are chosen randomly for sharp-colored noise u_0^2 and white noise u_0^3 as illustrated in Fig. 5.1 and which are then applied to nonlocal AC and CH models. Both regular potential and obstacle potential are considered.

5.3.1. Nonlocal Allen-Cahn equation

For the nonlocal AC equation for the regular and obstacle potentials, in Figs. 5.11 and 5.12 which correspond to the initial conditions u_0^2 and u_0^3 , respectively, we plot the predicted solutions obtained by NPF-Net and the corresponding prediction errors at the times $t = 5, 10, 50, 150$ (left to right) and, from top to bottom, the simulation solutions and errors at $\delta = 0.05$ and 0.1 . In the process of grain coarsening, one phase gradually becomes smaller and smaller and eventually vanishes in time. In Figs. 5.13 and 5.14, the evolution of the energy of the predicted solution with regular and obstacle potential are plotted respectively. We observe that in both cases the NN preserved the energy dissipation over time.

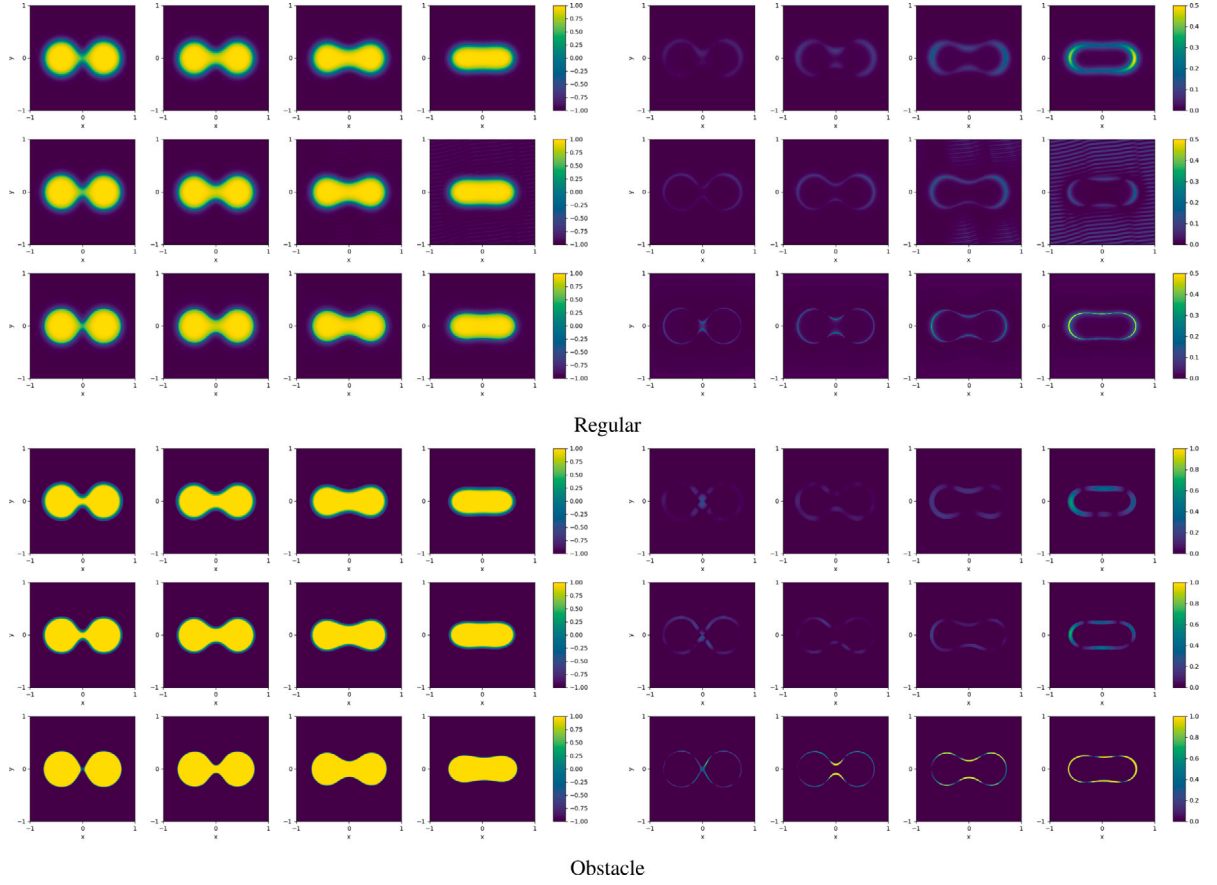


Fig. 5.10. Plots of the NPF-Net prediction solutions (left columns) and the corresponding numerical errors (right columns) for solving the nonlocal CH equation with regular and obstacle potentials in the two-dimensional merging bubbles example. The rows, from top to bottom, correspond to results at different values of $\delta = 0.05, 0.075, 0.095$. The columns display the results at four different times $t = 0.5, 1, 2, 5$.

5.3.2. Nonlocal Cahn–Hilliard equation

For the nonlocal CH equation for the regular and obstacle potentials, in Fig. 5.15 which corresponds to the sharp-colored noise initial condition u_0^2 , we plot the predicted solutions obtained by NPF-Net and the corresponding prediction errors at the times $t = 0.5, 1, 2, 5$ (left to right) and the two values of $\delta = 0.05, 0.095$ (top to bottom). We observe a slight increase in errors over time for the obstacle potential function. However, despite the sharpness of the solution, our network model effectively learns and captures this behavior, preserving the sharp interfaces throughout the simulation. On Fig. 5.16 we also plot the evolution for the nonlocal energy (2.2). We observe that the NN approach preserves energy decay over time for both regular and obstacle potential functions.

5.4. Three-dimensional nonlocal Allen–Cahn equation

To demonstrate the capability of our model, we apply the proposed method to 3D nonlocal AC problems with an interfacial parameter of $\epsilon = 0.05$. The same network architectures and training strategies used for the 2D problems are extended to construct NPF-Net for 3D problems. Consequently, the input U_n is now a $1 \times N_x \times N_y \times N_z$ tensor, and the filter size in each convolutional layer changes to $K \times K \times K$ with $K = 3$. We set $\delta = 0.1$, $\Delta t = 0.1$, and $T_{train} = 10$, while the spatial mesh size is set to $h = 2/64$. To test NPF-Net for solving 3D nonlocal problems, we consider two types of initial conditions. One is the star-shaped initial condition, and the other is the sharp-colored noise initial condition. Fig. 5.17 shows the iso-surface (value 0) of the predicted solutions and relative L_2 errors by NPF-Net at time instances $t = 0, 10, 50, 100$ for the nonlocal AC equation with both regular and obstacle potential functions. The results clearly illustrate the grain coarsening process in both cases. Due to the sharper interface in the obstacle potential function, the error is larger compared to the regular potential function.

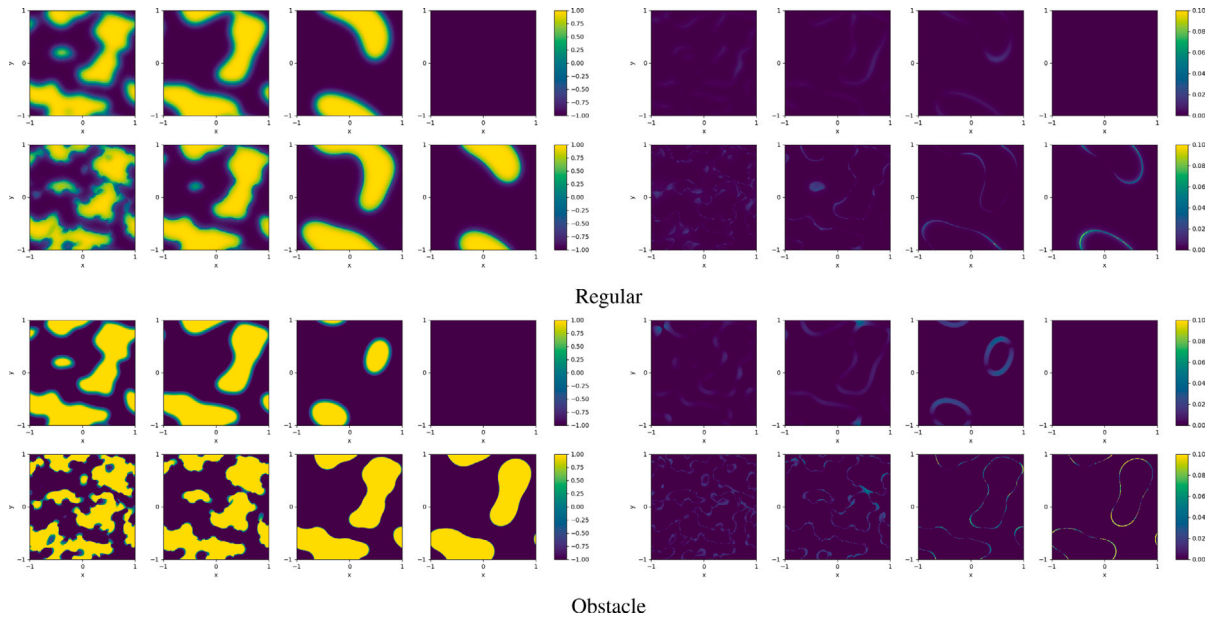


Fig. 5.11. Plots of the NPF-Net prediction solutions (left columns) and the corresponding numerical errors (right columns) for solving the nonlocal AC equation with regular and obstacle potentials in the 2D grain coarsening example. Sharp-colored noise initial condition is used. The rows, from Top to bottom, correspond to results at different values of $\delta = 0.05, 0.1$. The columns display the results at four different times $t = 5, 10, 50, 150$.

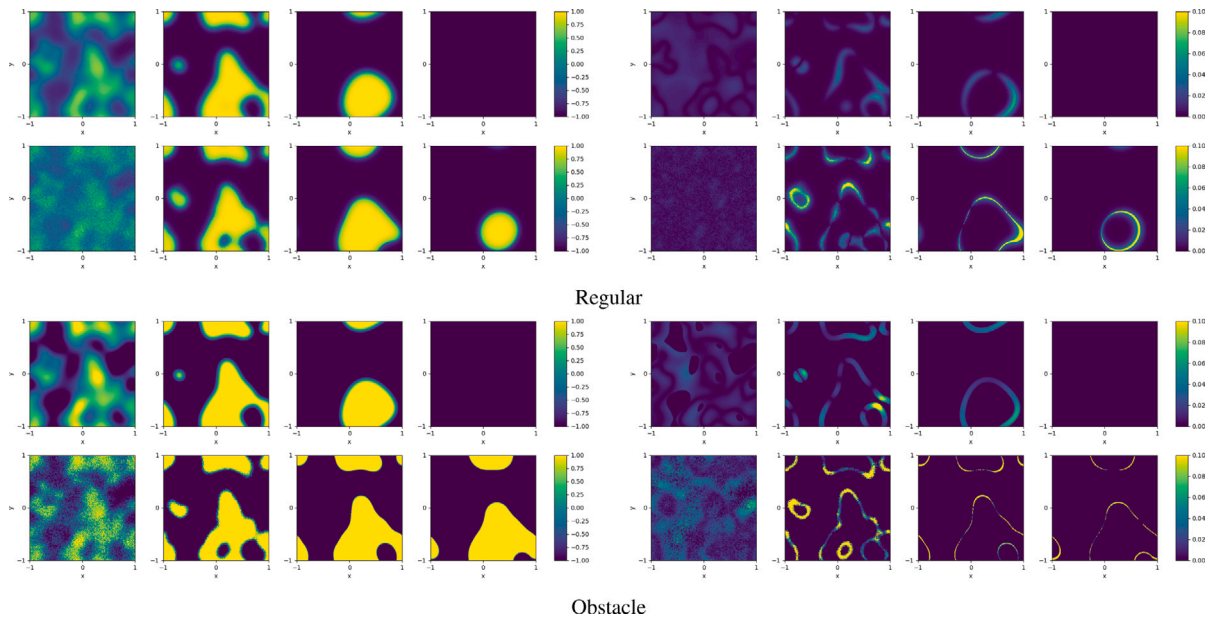


Fig. 5.12. Plots of the NPF-Net prediction solutions (left columns) and the corresponding numerical errors (right columns) for solving the nonlocal AC equation with regular and obstacle potentials in the 2D grain coarsening example. White noise initial condition is used. The rows, from Top to bottom, correspond to results at different values of $\delta = 0.05, 0.1$. The columns display the results at four times $t = 5, 10, 50, 150$.

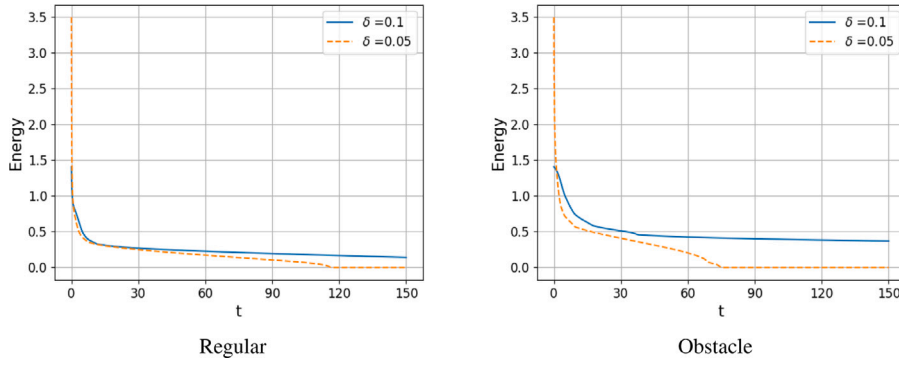


Fig. 5.13. Evolution of the nonlocal energy (2.2) for the solution of nonlocal AC equation obtained by the NN simulations with sharp-colored noise initial condition, ($\epsilon = 0.05$, $N_x = N_y = 256$, and $\Delta t = 0.1$).

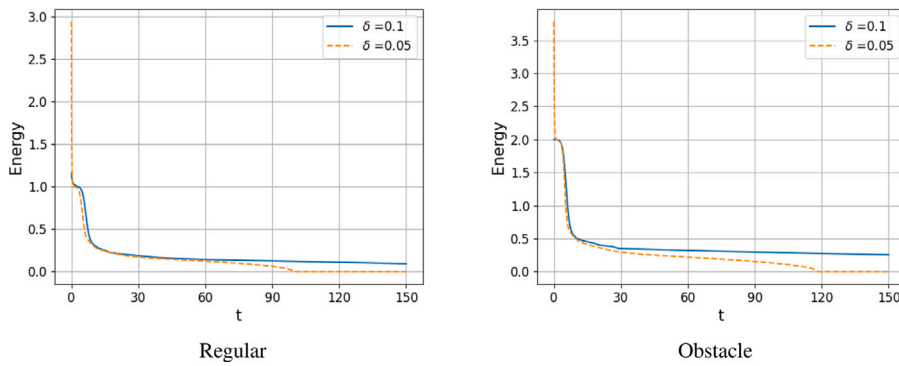


Fig. 5.14. Evolution of the nonlocal energy (2.2) for the solution of nonlocal AC equation obtained by the NN simulations with white noise initial condition, ($\epsilon = 0.05$, $N_x = N_y = 256$, and $\Delta t = 0.1$).

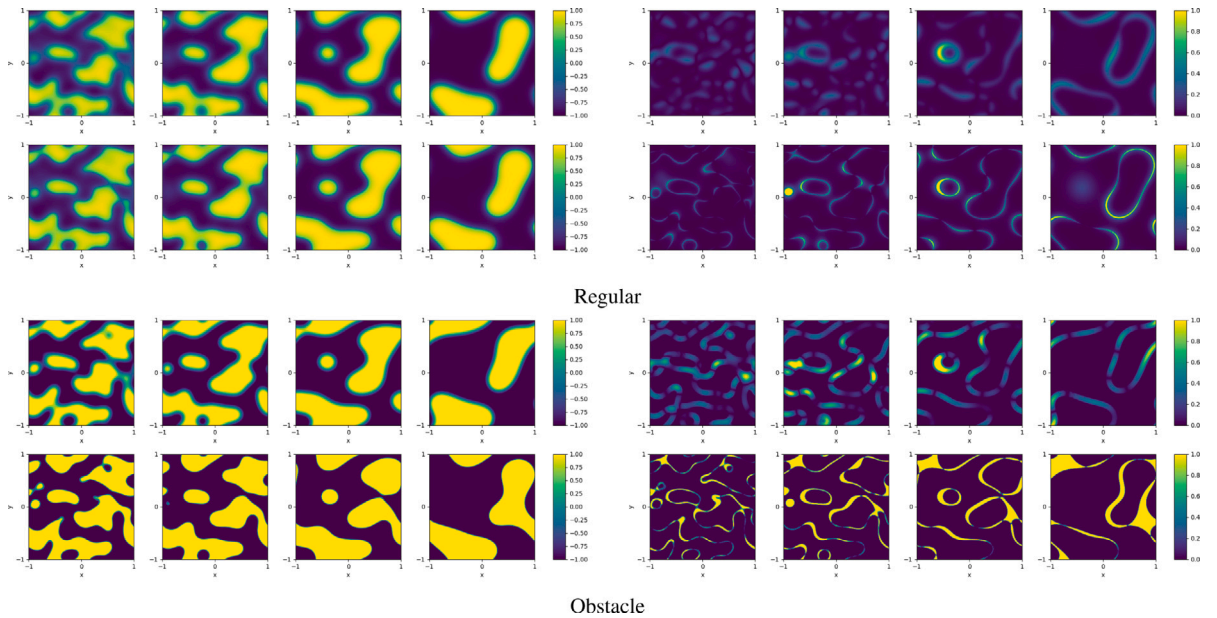


Fig. 5.15. Plots of the NPF-Net prediction solutions (left columns) and the corresponding numerical errors (right columns) for solving the nonlocal CH equation with regular and obstacle potentials in the 2D grain coarsening example. Sharp-colored noise initial condition is used. From Top to bottom, the rows correspond to results at different values of $\delta = 0.05, 0.095$. The columns display the results at four different times $t = 0.5, 1, 2, 5$.

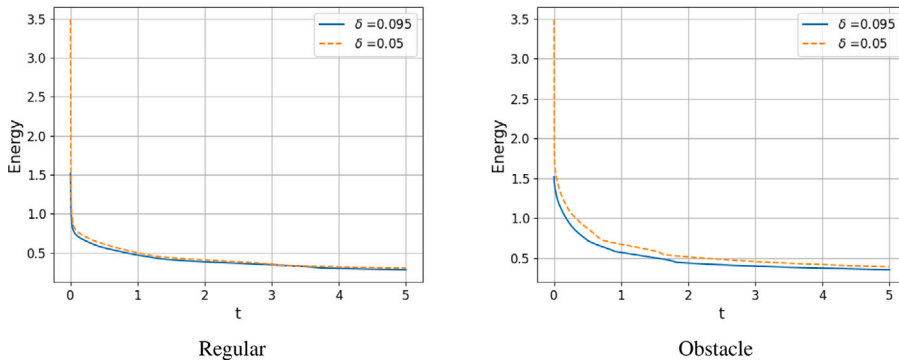


Fig. 5.16. Evolution of the nonlocal energy (2.2) for the solution of nonlocal CH equation obtained by the NN simulations with sharp-colored noise initial condition, ($\epsilon = 0.05$, $N_x = N_y = 256$, and $\Delta t = 0.01$).

6. Concluding remarks

This paper introduces NPF-Net, an end-to-end deep learning method for solving nonlocal Allen–Cahn (AC) and Cahn–Hilliard (CH) phase-field models. Our approach effectively handles both diffuse and sharp interfaces, learns fully discrete operators without ground-truth data, and demonstrates excellent performance across several potential functions and initial conditions. The time-adaptive training strategy significantly reduces training time while maintaining accuracy and the model shows strong generalization capabilities. Our method is the incorporation of a bound limiter in the network architecture and the use of projection formulas in the loss function. These features allow NPF-Net to efficiently handle non-smooth potential functions and capture sharp interfaces which are challenging aspects in both traditional numerical methods and existing machine learning approaches. This capability enables more realistic modeling of complex material behaviors, particularly for systems with distinct phase boundaries or discontinuous processes. Despite their promising performance, NPF-Net can be improved in several aspects. For example, the current method is designed for a fixed horizon δ , which means that NPF-Net needs to be retrained once δ is changed. This limits the generalization of NPF-Net. To resolve this issue, we will treat δ and/or other parameters in the target equation as additional inputs to the NPF-Net model such that the trained NPF-Net, as a function of δ , can be used to solve the nonlocal AC and CH equations with different horizons. The network architecture and the training strategy need to be significantly modified to ensure the model can learn the relationship between δ and the solution dynamics across a wide range of δ values. Moreover, we will extend the method to solve nonlocal mass-conserving Cahn–Hilliard equations which are crucial for many physical applications.

The NPF-Net strategy we have developed for the nonlocal AC and CH settings can be applied to many other applications involving phase-field model. Because of its flexibility and scope, it is natural to expect that the gains effected by NPF-Net for the AC and CH settings could extend to other applications. To provide further evidence about the improvement gains of NPF-Net, we will, in the future, apply that approach to other applications for which we have in hand nonlocal models and their discretizations and which incorporate phase fields. Specifically, we will first consider alloy solidification models [21,22,41] and superconductivity models [42–45] to which we apply NPF-Net methodology. These activities will provide further evidence about the gains effected by using that methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Applied Mathematics program, under the contracts ERKJ388 and ERKJ443. ORNL is operated by UT-Battelle, LLC., for the U.S. Department of Energy under Contract DE-AC05-00OR22725. Lili Ju and Max Gunzburger acknowledge the support from the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research program under grants DE-SC0022254, DE-SC0023171 and DE-SC0025527, respectively.

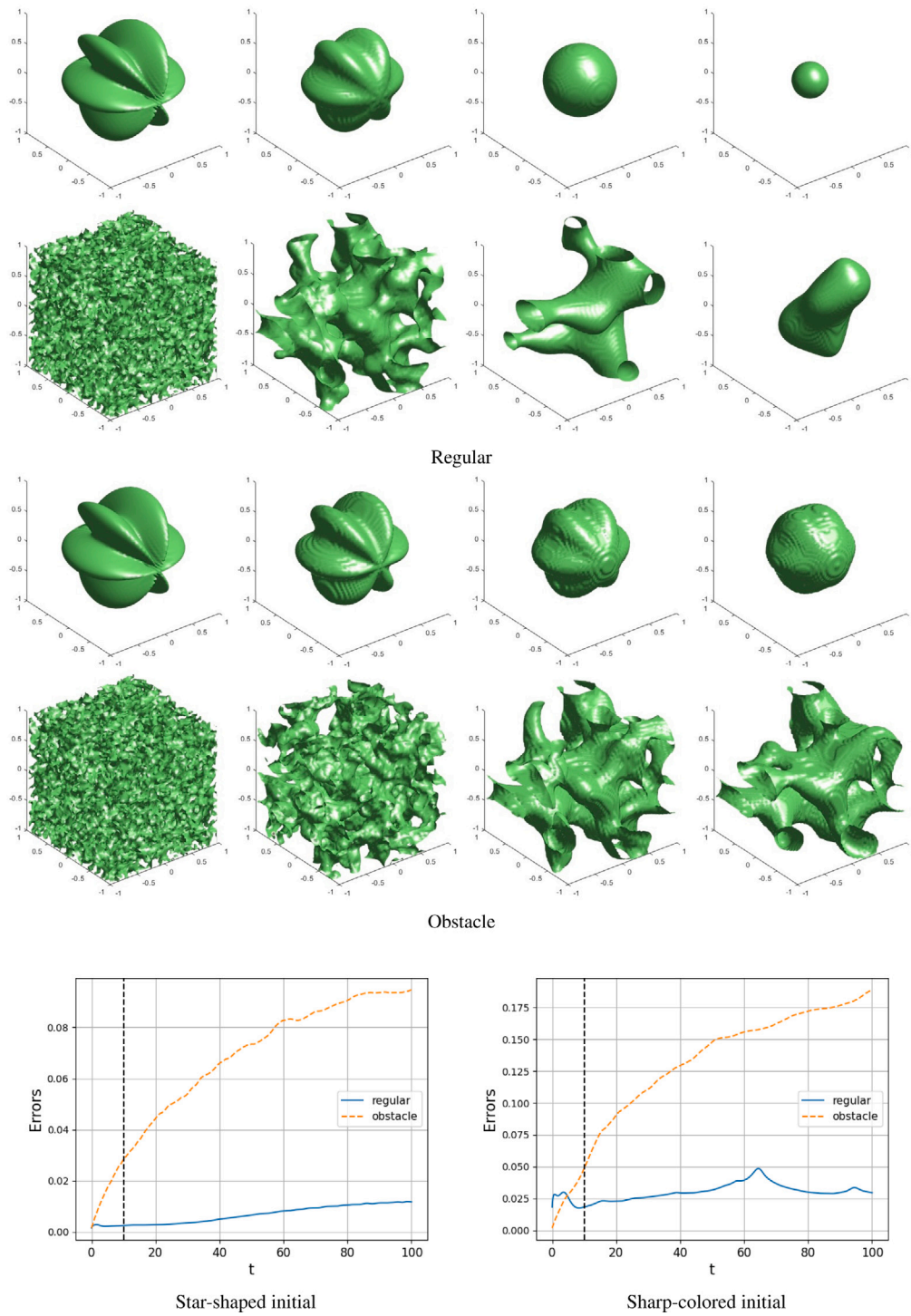


Fig. 5.17. The NPF-Net prediction solutions and relative L_2 errors for solving nonlocal AC equation with regular and obstacle potential function under two different types of initial conditions. Top four rows: plots of the iso-surface (value 0) of the predicted solution at times $T = 0, 10, 50, 100$; Bottom row: evolution of the relative L_2 errors of the predicted solution.

References

- [1] R. Kobayashi, Modeling and numerical simulations of dendritic crystal growth, *Physica D* 63 (3–4) (1993) 410–423.
- [2] S.-L. Wang, R. Sekerka, A. Wheeler, B. Murray, S. Coriell, R. Braun, G. McFadden, Thermodynamically-consistent phase-field models for solidification, *Physica D* 69 (1–2) (1993) 189–200.
- [3] P. Diehl, R. Lipton, T. Wick, M. Tyagi, A comparative review of peridynamics and phase-field models for engineering fracture mechanics, *Comput. Mech.* 69 (6) (2022) 1259–1293.
- [4] A.L. Bertozzi, S. Esedoglu, A. Gillette, inpainting of binary images using the Cahn–Hilliard equation, *IEEE Trans. Image Process.* 16 (1) (2006) 285–291.
- [5] H. Garcke, K.F. Lam, V. Styles, Cahn–Hilliard inpainting with the double obstacle potential, *SIAM J. Imaging Sci.* 11 (3) (2018) 2064–2089.
- [6] M. De Florio, E. Schiassi, F. Calabrò, R. Furfaro, Physics-Informed Neural Networks for 2nd order ODEs with sharp gradients, *J. Comput. Appl. Math.* 436 (2024) 115396.
- [7] A. Krishnapriyan, A. Gholami, S. Zhe, R. Kirby, M.W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Adv. Neural Inf. Process. Syst.* 34 (2021) 26548–26560.
- [8] Z. Mao, X. Meng, Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving partial differential equations with sharp solutions, *Appl. Math. Mech.* 44 (7) (2023) 1069–1084.
- [9] H. You, Y. Yu, M. D’Elia, T. Gao, S. Silling, Nonlocal kernel network (NKN): A stable and resolution-independent deep neural network, *J. Comput. Phys.* 469 (2022) 111536.
- [10] C.L. Zhao, Solving Allen–Cahn and Cahn–Hilliard equations using the adaptive physics informed neural networks, *Commun. Comput. Phys.* 29 (3) (2020).
- [11] B. Lin, Z. Mao, Z. Wang, G.E. Karniadakis, Operator learning enhanced physics-informed neural networks for solving partial differential equations characterized by sharp solutions, 2023, arXiv preprint arXiv:2310.19590.
- [12] S. Goswami, A. Bora, Y. Yu, G.E. Karniadakis, Physics-informed deep neural operator networks, in: *Machine Learning in Modeling and Simulation: Methods and Applications*, Springer, 2023, pp. 219–254.
- [13] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, Neural operator: Learning maps between function spaces with applications to pdes, *J. Mach. Learn. Res.* 24 (89) (2023) 1–97.
- [14] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229.
- [15] Y. Yu, N. Liu, F. Lu, T. Gao, S. Jafarzadeh, S. Silling, Nonlocal attention operator: Materializing hidden knowledge towards interpretable physics discovery, 2024, arXiv preprint arXiv:2408.07307.
- [16] V. Oommen, K. Shukla, S. Goswami, R. Dingreville, G.E. Karniadakis, Learning two-phase microstructure evolution using neural operators and autoencoder architectures, *npj Comput. Mater.* 8 (1) (2022) 190.
- [17] E. Bretin, R. Denis, S. Masnou, G. Terii, Learning phase field mean curvature flows with neural networks, *J. Comput. Phys.* 470 (2022) 111579.
- [18] W. Li, M.Z. Bazant, J. Zhu, Phase-Field DeepONet: Physics-informed deep operator neural network for fast simulations of pattern formation governed by gradient flows of free-energy functionals, *Comput. Methods Appl. Mech. Engrg.* 416 (2023) 116299.
- [19] Y. Geng, Y. Teng, Z. Wang, L. Ju, A deep learning method for the dynamics of classic and conservative Allen–Cahn equations based on fully-discrete operators, *J. Comput. Phys.* 496 (2024) 112589.
- [20] E. Davoli, L. Scarpa, L. Trussardi, Local asymptotics for nonlocal convective Cahn–Hilliard equations with $W^{1,1}$ kernel and singular potential, *J. Differential Equations* 289 (2021) 35–58.
- [21] O. Burkovska, M. Gunzburger, On a nonlocal Cahn–Hilliard model permitting sharp interfaces, *Math. Models Methods Appl. Sci.* 31 (09) (2021) 1749–1786.
- [22] O. Burkovska, Non-isothermal nonlocal phase-field models with a double-obstacle potential, 2023, arXiv preprint arXiv:2310.07861.
- [23] Q. Du, L. Ju, X. Li, Z. Qiao, Maximum principle preserving exponential time differencing schemes for the nonlocal Allen–Cahn equation, *SIAM J. Numer. Anal.* 57 (2) (2019) 875–898.
- [24] Q. Du, J. Yang, Asymptotically compatible Fourier spectral approximations of nonlocal Allen–Cahn equations, *SIAM J. Numer. Anal.* 54 (3) (2016) 1899–1919.
- [25] Q. Du, J. Yang, Fast and accurate implementation of Fourier spectral approximations of nonlocal diffusion operators and its applications, *J. Comput. Phys.* 332 (2017) 118–134.
- [26] O. Burkovska, I. Mustapha, An efficient proximal-based approach for solving nonlocal Allen–Cahn equations, 2024, Preprint, arXiv:2410.06455.
- [27] S. Boyd, N. Parikh, Proximal algorithms, *Found. Trends Optim.* 1 (3) (2013) 123–231.
- [28] R. Witula, D. Ślota, Cardano’s formula, square roots, Chebyshev polynomials and radicals, *J. Math. Anal. Appl.* 363 (2) (2010) 639–647.
- [29] Q. Du, L. Ju, X. Li, Z. Qiao, Stabilized linear semi-implicit schemes for the nonlocal Cahn–Hilliard equation, *J. Comput. Phys.* 363 (2018) 39–54.
- [30] Z. Guan, J. Lowengrub, C. Wang, Convergence analysis for second-order accurate schemes for the periodic nonlocal Allen–Cahn and Cahn–Hilliard equations, *Math. Methods Appl. Sci.* 40 (18) (2017) 6836–6863.
- [31] Z. Guan, J.S. Lowengrub, C. Wang, S.M. Wise, Second order convex splitting schemes for periodic nonlocal Cahn–Hilliard and Allen–Cahn equations, *J. Comput. Phys.* 277 (2014) 48–71.
- [32] P.W. Bates, S. Brown, J. Han, Numerical analysis for a nonlocal Allen–Cahn equation, *Int. J. Numer. Anal. Model.* 6 (1) (2009) 33–49.
- [33] M. Ainsworth, Z. Mao, Analysis and approximation of a fractional Cahn–Hilliard equation, *SIAM J. Numer. Anal.* 55 (4) (2017) 1689–1718.
- [34] G. Acosta, F.M. Bereseteche, Numerical approximations for a fully fractional Allen–Cahn equation, *ESAIM Math. Model. Numer. Anal.* 55 (2021) S3–S28.
- [35] Q. Du, X. Feng, Chapter 5 - The phase field method for geometric moving interfaces and their numerical approximations, in: A. Bonito, R.H. Nochetto (Eds.), *Geometric Partial Differential Equations - Part I*, in: *Handbook of Numerical Analysis*, vol. 21, Elsevier, 2020, pp. 425–508.
- [36] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [37] S. Albawi, T.A. Mohammed, S. Al-Zawi, Understanding of a convolutional neural network, in: *2017 International Conference on Engineering and Technology, ICET*, 2017, pp. 1–6.
- [38] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Y. Bengio, Y. LeCun (Eds.), *3rd International Conference on Learning Representations, ICLR 2015*, San Diego, CA, USA, May 7–9, 2015, Conference Track Proceedings, 2015.
- [39] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, A. Lerer, Automatic differentiation in pytorch, in: *NIPS 2017 Workshop on Autodiff*, 2017.
- [40] S. Müller, L. Schüller, A. Zech, F. Heße, GSTools v1.3: a toolbox for geostatistical modelling in Python, *Geosci. Model Dev.* 15 (7) (2022) 3161–3182.
- [41] P. Khodabakhshi, O. Burkovska, K. Willcox, M. Gunzburger, Multifidelity methods for uncertainty quantification of a nonlocal model for phase changes in materials, *Comput. Struct.* 34 (2023) 1306–1331.
- [42] J. Chapman, Q. Du, M. Gunzburger, A Ginzburg–Landau type model of superconducting/normal junctions including Josephson junctions, *European J. Appl. Math.* 6 (1995) 97–114.
- [43] J. Chapman, Q. Du, M. Gunzburger, A model for variable thickness superconducting thin films, *Z. Angew. Math. Phys.* 47 (1996) 410–431.
- [44] J. Deang, Q. Du, M. Gunzburger, J. Peterson, A model for variable thickness superconducting thin films, *Philos. Trans. R. Soc. Lond. A* 355 (1997) 1957–1968.
- [45] Q. Du, M. Gunzburger, J. Peterson, Analysis and approximation of the Ginzburg–Landau model of superconductivity, *SIAM Rev.* 34 (1992) 54–81.